

DeepSeek-V3 Technical Report

DeepSeek-AI

research@deepseek.com

Abstract

We present DeepSeek-V3, a strong Mixture-of-Experts (MoE) language model with 671B total parameters with 37B activated for each token. To achieve efficient inference and cost-effective training, DeepSeek-V3 adopts Multi-head Latent Attention (MLA) and DeepSeekMoE architectures, which were thoroughly validated in DeepSeek-V2. Furthermore, DeepSeek-V3 pioneers an auxiliary-loss-free strategy for load balancing and sets a multi-token prediction training objective for stronger performance. We pre-train DeepSeek-V3 on 14.8 trillion diverse and high-quality tokens, followed by Supervised Fine-Tuning and Reinforcement Learning stages to fully harness its capabilities. Comprehensive evaluations reveal that DeepSeek-V3 outperforms other open-source models and achieves performance comparable to leading closed-source models. Despite its excellent performance, DeepSeek-V3 requires only 2.788M H800 GPU hours for its full training. In addition, its training process is remarkably stable. Throughout the entire training process, we did not experience any irrecoverable loss spikes or perform any rollbacks. The model checkpoints are available at <https://github.com/deepseek-ai/DeepSeek-V3>.

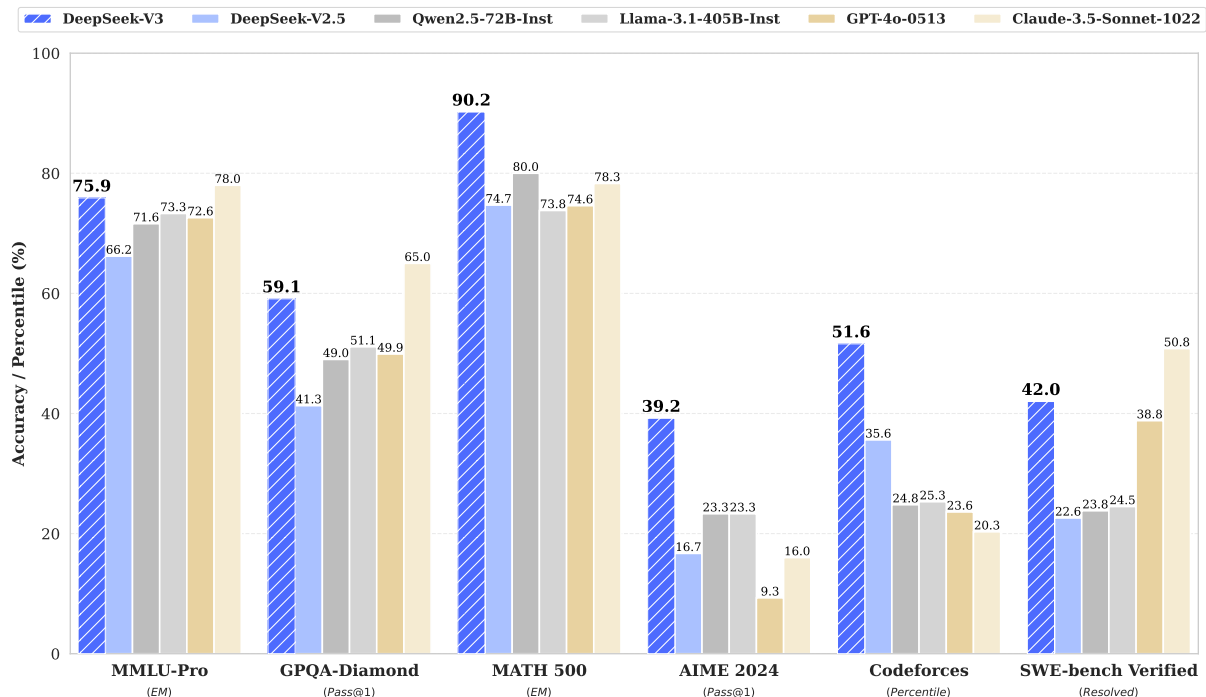


Figure 1 | Benchmark performance of DeepSeek-V3 and its counterparts.

Contents

1	Introduction	4
2	Architecture	6
2.1	Basic Architecture	6
2.1.1	Multi-Head Latent Attention	7
2.1.2	DeepSeekMoE with Auxiliary-Loss-Free Load Balancing	8
2.2	Multi-Token Prediction	10
3	Infrastructures	11
3.1	Compute Clusters	11
3.2	Training Framework	12
3.2.1	DualPipe and Computation-Communication Overlap	12
3.2.2	Efficient Implementation of Cross-Node All-to-All Communication	13
3.2.3	Extremely Memory Saving with Minimal Overhead	14
3.3	FP8 Training	14
3.3.1	Mixed Precision Framework	15
3.3.2	Improved Precision from Quantization and Multiplication	16
3.3.3	Low-Precision Storage and Communication	18
3.4	Inference and Deployment	18
3.4.1	Prefilling	19
3.4.2	Decoding	19
3.5	Suggestions on Hardware Design	20
3.5.1	Communication Hardware	20
3.5.2	Compute Hardware	20
4	Pre-Training	21
4.1	Data Construction	21
4.2	Hyper-Parameters	22
4.3	Long Context Extension	23
4.4	Evaluations	24
4.4.1	Evaluation Benchmarks	24
4.4.2	Evaluation Results	24
4.5	Discussion	26
4.5.1	Ablation Studies for Multi-Token Prediction	26
4.5.2	Ablation Studies for the Auxiliary-Loss-Free Balancing Strategy	26

4.5.3	Batch-Wise Load Balance VS. Sequence-Wise Load Balance	27
5	Post-Training	28
5.1	Supervised Fine-Tuning	28
5.2	Reinforcement Learning	29
5.2.1	Reward Model	29
5.2.2	Group Relative Policy Optimization	30
5.3	Evaluations	30
5.3.1	Evaluation Settings	30
5.3.2	Standard Evaluation	31
5.3.3	Open-Ended Evaluation	33
5.3.4	DeepSeek-V3 as a Generative Reward Model	33
5.4	Discussion	34
5.4.1	Distillation from DeepSeek-R1	34
5.4.2	Self-Rewarding	34
5.4.3	Multi-Token Prediction Evaluation	35
6	Conclusion, Limitations, and Future Directions	35
A	Contributions and Acknowledgments	45
B	Ablation Studies for Low-Precision Training	47
B.1	FP8 v.s. BF16 Training	47
B.2	Discussion About Block-Wise Quantization	47
C	Expert Specialization Patterns of the 16B Aux-Loss-Based and Aux-Loss-Free Models	48

1. 引言

近年来，大型语言模型（LLMs）经历了快速的迭代与演进（Anthropic, 2024年; Google, 2024年; OpenAI, 2024a），持续缩小与通用人工智能（AGI）之间的差距。除了闭源模型之外，开源模型——包括 DeepSeek 系列（DeepSeek-AI, 2024a,b,c; Guo et al., 2024年）、LLaMA 系列（AI @Meta, 2024a,b; Touvron et al., 2023a,b）、Qwen 系列（Qwen, 2023年, 2024a,b）以及 Mistral 系列（Jiang et al., 2023年; Mistral, 2024年）——也在不断取得重要进展，努力缩小与闭源对应模型之间的差距。为进一步推动开源模型能力的边界，我们对模型进行缩放并推出 DeepSeek-V3，这是一种拥有 671B 参数的大型专家混合（MoE）模型，其中每个标记激活 37B。

以前瞻性的视角，我们始终致力于实现强劲模型性能与经济的成本。因此，在架构方面，DeepSeek-V3 仍然采用多头潜在注意（MLA）（DeepSeek-AI, 2024c）以实现高效推理，并采用 DeepSeekMoE（Dai 等人, 2024年）以实现具有成本效益的训练。这两种架构已在 DeepSeek-V2（DeepSeek-AI, 2024c）中得到验证，证明其能够在实现高效训练与推理的同时保持稳健的模型性能。在基本架构之外，我们实施了两项额外策略，以进一步提升模型能力。首先，DeepSeek-V3 首创一种用于负载均衡的无辅助损失策略（Wang 等人, 2024a），旨在尽量减少为促进负载均衡而对模型性能造成的不利影响。其次，DeepSeek-V3 采用多标记预测训练目标函数，我们观察到这可以提升评测基准上的整体性能。

为了实现高效训练，我们支持 FP8 混合精度训练，并对训练框架进行了全面优化。低精度训练已成为实现高效训练的有前景方案（Dettmers 等, 2022年; Kalamkar 等, 2019年; Narang 等, 2017年; Peng 等, 2023b），其演进与硬件能力的进步密切相关（Luo 等, 2024年; Micikevicius 等, 2022年; Rouhani 等, 2023a）。在本工作中，我们提出了一个 FP8 混合精度训练框架，并首次在一个极其大规模的模型上验证了其有效性。通过对 FP8 计算和存储的支持，我们既实现了训练加速，又降低了 GPU 内存使用。对于训练框架，我们为高效的流水线并行设计了 DualPipe 算法，它具有更少的流水线气泡，并通过计算-通信重叠在训练过程中隐藏了大部分通信。该重叠确保随着模型进一步缩放，只要我们保持恒定的计算与通信比，就仍然可以在跨节点采用细粒度专家，同时实现近乎为零的全对全通信开销。此外，我们还开发了高效的跨节点全对全通信内核，以充分利用 InfiniBand（IB）和 NVLink 带宽。进一步地，我们还精细优化了内存占用，使得无需采用代价高昂的张量并行也能训练 DeepSeek-V3。综合以上措施，我们实现了高训练效率。

在预训练期间，我们在 14.8T 高质量且多样化的 Token 上训练 DeepSeek-V3。预训练过程异常稳定。在整个训练过程中，我们未遇到任何不可恢复的损失突增，也无需回滚。接着，我们为 DeepSeek-V3 进行两阶段的上下文长度扩展：第一阶段将最大上下文长度扩展到 32K，第二阶段进一步扩展到 128K。随后，我们对 DeepSeek-V3 的基础模型开展后训练，包括 Supervised Fine-Tuning (SFT) 和 Reinforcement Learning (RL)，以使其与人类偏好对齐并进一步释放其潜力。在后训练阶段，我们从 DeepSeek-R1 系列模型中蒸馏其推理能力，同时谨慎地在模型准确率

训练成本	预训练	上下文扩展	后训练	总计
以 H800 GPU 小时计	2664K	119K	5K	2788K
以美元计	\$5.328M	\$0.238M	\$0.01M	\$5.576M

表 1 | DeepSeek-V3 的训练成本，假设 H800 的租赁价格为每 GPU 小时 \$2

与生成长度之间的平衡。

我们在一系列全面的基准上评估了 DeepSeek-V3。尽管其训练成本经济，全面评测表明 DeepSeek-V3-Base 已成为当前最强的开源基础模型，尤其在代码与数学方面。其聊天版本也优于其他开源模型，并在一系列标准与开放式基准上取得可与领先的闭源模型（包括 GPT-4o 和 Claude-3.5-Sonnet）相当的性能。

最后，我们再次强调 DeepSeek-V3 的经济性训练成本（见表1），这得益于我们在算法、框架与硬件上的优化协同设计。在预训练阶段，DeepSeek-V3 每处理一万亿 Token 仅需 180K 个 H800 GPU 小时，即在我们配备 2048 张 H800 GPU 的集群上为 3.7 天。因此，我们的预训练阶段在不到两个月内完成，耗费 2664K GPU 小时。再加上用于上下文长度扩展的 119K GPU 小时和用于后训练的 5K GPU 小时，DeepSeek-V3 的完整训练仅需 2.788M GPU 小时。假设 H800 GPU 的租赁价格为每 GPU 小时 \$2，我们的总训练成本仅为 \$5.576M。请注意，以上成本仅包括 DeepSeek-V3 的正式训练，不包括与在架构、算法或数据上的前期研究与消融实验相关的成本。

我们的主要贡献包括：

架构：创新的负载均衡策略与训练目标函数

- 在 DeepSeek-V2 的高效架构之上，我们首创了一种用于负载均衡的无辅助损失策略，可最小化为促进负载均衡而产生的性能下降。
- 我们研究了多标记预测（MTP）目标函数，并证明其有助于提升模型性能。它还可用于推测式解码，以实现推理加速。

预训练：迈向极致训练效率

- 我们设计了一个 FP8 混合精度训练框架，并首次在一个极其大规模的模型上验证了 FP8 训练的可行性和有效性。
- 通过算法、框架与硬件的协同设计，我们克服了跨节点 MoE 训练中的通信瓶颈，实现了接近完全的计算-通信重叠。这大幅提升了我们的训练效率并降低了训练成本，使我们得以在无需额外开销的情况下进一步缩放模型规模。
- 以仅 2.664M 个 H800 GPU 小时的成本，我们在 14.8T Token 上完成了 DeepSeek-V3 的预训练，产出了当前最强的开源基础模型。预训练之后的后续训练阶段仅需 0.1M GPU 小时。

后训练：来自 DeepSeek-R1 的知识蒸馏

- 我们提出了一种创新的方法，将长链式思维（CoT）模型——具体而言，来自 DeepSeek R1 系列中的某个模型——的推理能力蒸馏到标准的 LLMs 中，尤其是 DeepSeek-V3。我们的流程优雅地将

R1 的验证与反思模式融入 DeepSeek-V3，并显著提升其推理性能。同时，我们也维持对 DeepSeek-V3 的输出风格和长度的控制。

核心评测结果总结

- **知识：**(1) 在 MMLU、MMLU-Pro 和 GPQA 等教育类基准上，DeepSeek-V3 优于所有其他开源模型，分别在 MMLU、MMLU-Pro 和 GPQA 上取得 88.5、75.9 和 59.1 的成绩。其性能可与 GPT-4o 和 Claude-Sonnet-3.5 等领先的闭源模型相媲美，缩小了该领域开源与闭源模型之间的差距。(2) 在事实性基准上，DeepSeek-V3 在 SimpleQA 与 Chinese SimpleQA 两项上均在开源模型中展现出更优性能。尽管它在英文事实知识（SimpleQA）上不及 GPT-4o 和 Claude-Sonnet-3.5，但在中文事实知识（Chinese SimpleQA）上则超越了这些模型，凸显了其在中文事实知识方面的优势。
- **代码、数学与推理：**(1) 在所有 non-long-CoT 的开源与闭源模型中，DeepSeek-V3 在数学相关基准上的性能达到了当前最先进水平。值得注意的是，它在 MATH-500 等特定基准上甚至超过了 o1-preview，展现出其强大的数学推理能力。(2) 在代码相关任务上，DeepSeek-V3 在 LiveCodeBench 等编程竞赛基准上成为性能最优的模型，巩固了其在该领域的领先地位。对于工程相关任务，尽管 DeepSeek-V3 的性能略低于 Claude-Sonnet-3.5，但仍以显著优势超过所有其他模型，展现出其在各类技术基准上的竞争力。

在本文余下部分，我们首先详细介绍我们的 DeepSeek-V3 模型架构（第2节）。随后，我们介绍我们的基础设施，包括我们的计算集群、训练框架、对 FP8 训练的支持、推理部署策略，以及我们对未来硬件设计的建议。接着，我们描述我们的预训练过程，包括训练数据的构建、超参数设置、长上下文扩展技术、相关评测，以及一些讨论（第4节）。之后，我们讨论我们的后训练工作，其中包括监督式微调（SFT）、强化学习（RL）、相应的评测与讨论（第5节）。最后，我们对本文进行总结，讨论 DeepSeek-V3 的现有局限性，并提出未来研究的潜在方向（第6节）。

2. 架构

我们首先介绍 DeepSeek-V3 的基本架构，其中，为了高效推理采用了 Multi-head Latent Attention (MLA) (DeepSeek-AI, 2024年c)，为了更经济的训练采用了 DeepSeekMoE (Dai 等, 2024年)。随后，我们提出一个 Multi-Token Prediction (MTP) 训练目标函数，我们观察到它能提升评测基准上的整体性能。对于文中未明确提及的其他次要细节，DeepSeek-V3 遵循 DeepSeek-V2 (DeepSeek-AI, 2024年c) 的设置。

2.1. 基本架构

DeepSeek-V3 的基本架构仍然在 Transformer 架构 (Vaswani 等, 2017 年) 的框架内。为实现高效的推理和经济的训练，DeepSeek-V3 也采用了 MLA 和 DeepSeekMoE，这两者已在 DeepSeek-V2 中得到充分验证。与 DeepSeek-V2 相比，例外之处在于我们另外引入了一种无需辅助损失的负载均衡

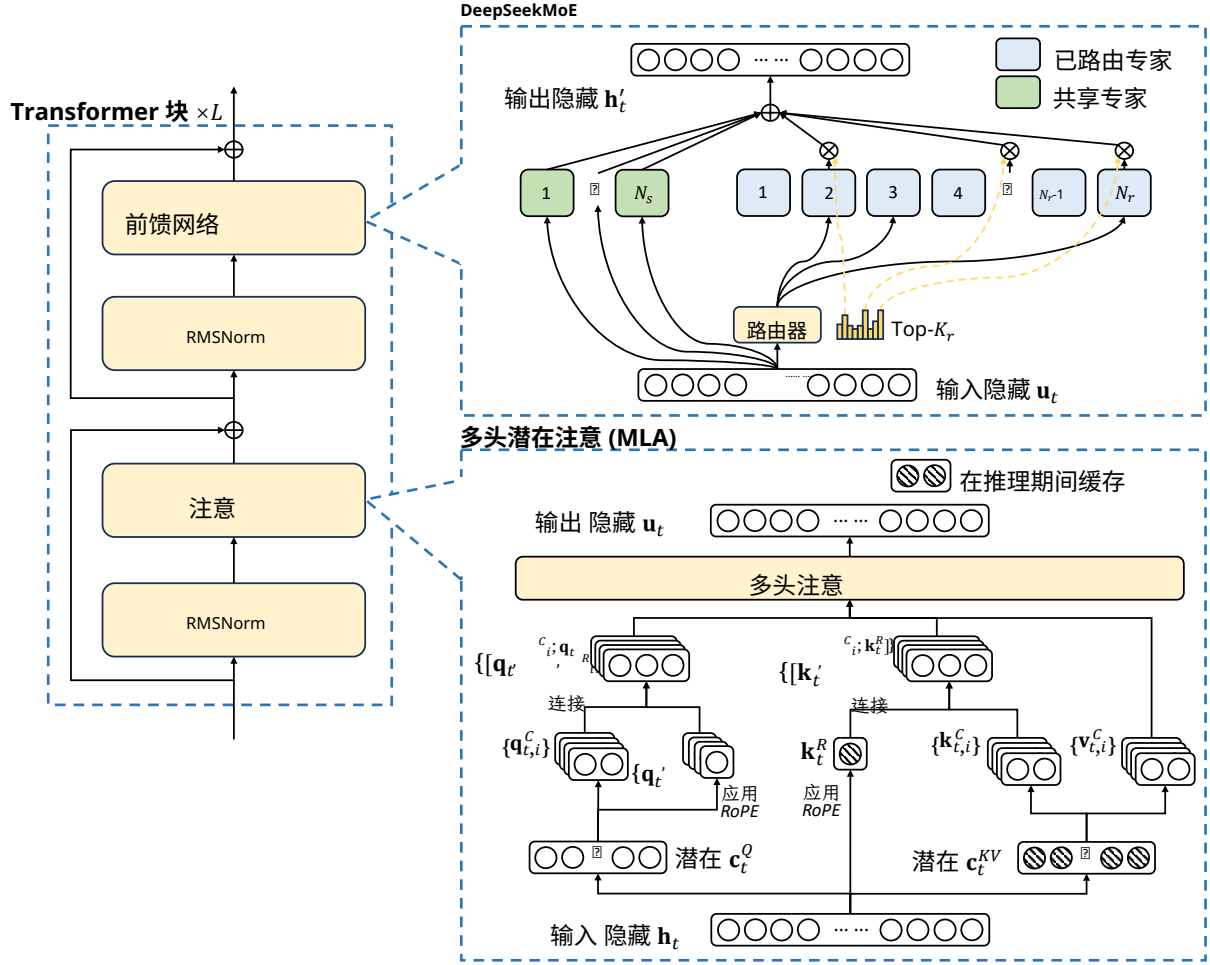


图 2 | DeepSeek-V3 的基本架构示意图。延续 DeepSeek-V2，我们采用 MLA 和 DeepSeekMoE，以实现高效推理和更经济的训练。

strategy (Wang 等人, 2024a) 用于 DeepSeekMoE，以缓解为确保负载均衡而带来的性能下降。图2 展示了 DeepSeek-V3 的基本架构，我们将在本节中简要回顾 MLA 和 DeepSeekMoE 的细节。

2.1.1. 多头潜在注意

对于注意，DeepSeek-V3 采用 MLA 架构。设 d 表示嵌入维度， n_h 表示注意头的数量， d_h 表示每个头的维度，且 $\mathbf{h}_t \in \mathbb{R}^d$ 表示在给定注意层中第 t 个标记的注意输入。MLA 的核心是针对注意键和值进行低秩联合压缩，以在推理期间减少键-值 (KV) 缓存：

$$\mathbf{c}_t^{KV} = W^{DKV} \mathbf{h}_t, \quad (1)$$

$$[\mathbf{k}_{t,1}^C; \mathbf{k}_{t,2}^C; \dots; \mathbf{k}_{t,n_h}^C] = \mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV}, \quad (2)$$

$$\mathbf{k}_t^R = \text{RoPE}(W^{KR} \mathbf{h}_t), \quad (3)$$

$$\mathbf{k}_{t,i} = [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R], \quad (4)$$

$$[\mathbf{v}_{t,1}^C; \mathbf{v}_{t,2}^C; \dots; \mathbf{v}_{t,n_h}^C] = \mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV}, \quad (5)$$

其中, $\mathbf{c}_t^{KV} \in \mathbb{R}^{d_c}$ 是键和值的压缩潜在向量; $d_c (\ll d_h n_h)$ 表示 KV 压缩维度; $W^{DKV} \in \mathbb{R}^{d_c \times d}$ 表示下投影矩阵; $W^{UK}, W^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ 分别为键和值的上投影矩阵; $W^{KR} \in \mathbb{R}^{d_h n_h \times d}$ 是用于生成携带 Rotary Positional Embedding (RoPE) (Su 等, 2024 年) 的解耦键的矩阵; $\text{RoPE}(\cdot)$ 表示应用 RoPE 矩阵的操作; 并且 $[\cdot; \cdot]$ 表示拼接。请注意, 对于 MLA, 仅需蓝色框中的向量 (即, $\mathbf{c}_t^{KV}$ 和 $\mathbf{k}_t^R$) 需要在生成过程中缓存, 这在显著减少 KV 缓存的同时, 仍能保持与标准多头注意 (MHA) (Vaswani 等, 2017 年) 相当的性能。

对于注意中的查询, 我们也执行低秩压缩, 这可以在训练期间减少激活内存:

$$\mathbf{c}_t^Q = W^{DQ} \mathbf{h}_t, \quad (6)$$

$$[\mathbf{q}_{t,1}^C; \mathbf{q}_{t,2}^C; \dots; \mathbf{q}_{t,n_h}^C] = \mathbf{q}_t^C = W^{UQ} \mathbf{c}_t^Q, \quad (7)$$

$$[\mathbf{q}_{t,1}^R; \mathbf{q}_{t,2}^R; \dots; \mathbf{q}_{t,n_h}^R] = \mathbf{q}_t^R = \text{RoPE}(W^{QR} \mathbf{c}_t^Q), \quad (8)$$

$$\mathbf{q}_{t,i} = [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R], \quad (9)$$

其中 $\mathbf{c}_t^Q \in \mathbb{R}^{d_c'}$ 是用于查询的压缩潜在向量; $d_c' (\ll d_h n_h)$ 表示查询压缩维度; $W^{DQ} \in \mathbb{R}^{d_c' \times d}$, $W^{UQ} \in \mathbb{R}^{d_h n_h \times d_c'}$ 分别是查询的下投影矩阵和上投影矩阵; 以及 $W^{QR} \in \mathbb{R}^{d_h n_h \times d_c'}$ 是用于生成携带 RoPE 的解耦查询的矩阵。

最终, 注意中的查询 ($\mathbf{q}_{t,i}$)、键 ($\mathbf{k}_{j,i}$) 和值 ($\mathbf{v}_{j,i}^C$) 被合并以得到最终的注意输出 $\mathbf{u}_t$:

$$\mathbf{o}_{t,i} = \sum_{j=1}^t \text{Softmax}_j \left(\frac{\mathbf{q}_{t,i}^T \mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}} \right) \mathbf{v}_{j,i}^C, \quad (10)$$

$$\mathbf{u}_t = W^O [\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \dots; \mathbf{o}_{t,n_h}], \quad (11)$$

其中 $W^O \in \mathbb{R}^{d \times d_h n_h}$ 表示输出投影矩阵。

2.1.2. 采用无辅助损失负载均衡的 **DeepSeek** 专家混合

DeepSeekMoE 的基本架构。 对于前馈网络 (FFN), DeepSeek-V3 采用 DeepSeekMoE 架构 (Dai 等, 2024 年)。与传统的专家混合架构 (如 GShard (Lepikhin 等, 2021 年)) 相比, DeepSeekMoE 使用更细粒度的专家, 并将部分专家划分为共享专家。设 $\mathbf{u}_t$ 表示第 t 个标记的前馈网络输入, 我们按如下方式计算前馈网络输出 $\mathbf{h}_t'$:

$$\mathbf{h}_t' = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t), \quad (12)$$

$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}}, \quad (13)$$

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases} \quad (14)$$

$$s_{i,t} = \text{Sigmoid}(\mathbf{u}_t^T \mathbf{e}_i), \quad (15)$$

其中, N_s 和 N_r 分别表示共享专家和已路由专家的数量; 前馈网络 $_i^{(s)}(\cdot)$ 和前馈网络 $_i^{(r)}(\cdot)$ 分别表示第 i 个共享专家和第 i 个已路由专家; K_r 表示被激活的已路由专家数量; $g_{i,t}$ 是第 i 个专家的门控值; $s_{i,t}$ 是标记到专家的亲和度; $\mathbf{e}_i$ 是第 i 个已路由专家的质心向量; 且 $\text{Topk}(\cdot, K)$ 表示集合, 包含为第 t 个标记与所有已路由专家计算的亲和度分数中最高 K 个分数。与 DeepSeek-V2 略有不同, DeepSeek-V3 使用 sigmoid 函数来计算亲和度分数, 并在所有选中的亲和度分数之间进行归一化以得到门控值。

无需辅助损失的负载均衡。 对于 MoE 模型, 不均衡的专家负载会导致路由坍塌 (Shazeer 等, 2017 年), 并在专家并行场景下降低计算效率。传统方案通常依赖辅助损失 (Fedus 等, 2021 年; Lepikhin 等, 2021 年) 来避免负载不均衡。然而, 过大的辅助损失会损害模型性能 (Wang 等人, 2024 年 a)。为在负载均衡与模型性能之间取得更好的权衡, 我们率先提出一种无辅助损失的负载均衡策略 (Wang 等人, 2024 年 a) 以确保负载均衡。具体而言, 我们为每个专家引入一个偏置项 b_i , 并将其加到对应的亲和度分数 $s_{i,t}$ 以确定 Top-k 路由:

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

请注意, 该偏置项仅用于路由。将与前馈网络输出相乘的门控数值仍然来源于原始亲和度分数 $s_{i,t}$ 。在训练期间, 我们持续监控每个训练步数的整批次专家负载。在每个步数结束时, 若对应专家过载, 我们将把偏置项减少 γ ; 若对应专家欠载, 则增加 γ , 其中 γ 是称为偏置更新速度的超参数。通过这种动态调整, DeepSeek-V3 在训练期间保持专家负载均衡, 并且相比仅通过纯辅助损失鼓励负载均衡的模型取得了更好的性能。

互补的序列级辅助损失。 尽管 DeepSeek-V3 主要依靠无辅助损失的策略来进行负载均衡, 但为防止任何单个序列内部出现极端不均衡, 我们还采用了一种互补的序列级平衡损失:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \quad (17)$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^T \mathbb{1}(s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r)), \quad (18)$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \quad (19)$$

$$P_i = \frac{1}{T} \sum_{t=1}^T s'_{i,t}, \quad (20)$$

其中平衡因子 α 是一个超参数, 在 DeepSeek-V3 中将被赋予一个极小的数值; $\mathbb{1}(\cdot)$ 表示指示函数; 以及 T 表示一个序列中的令牌数量。序列级平衡损失鼓励每个序列上的专家负载保持平衡。

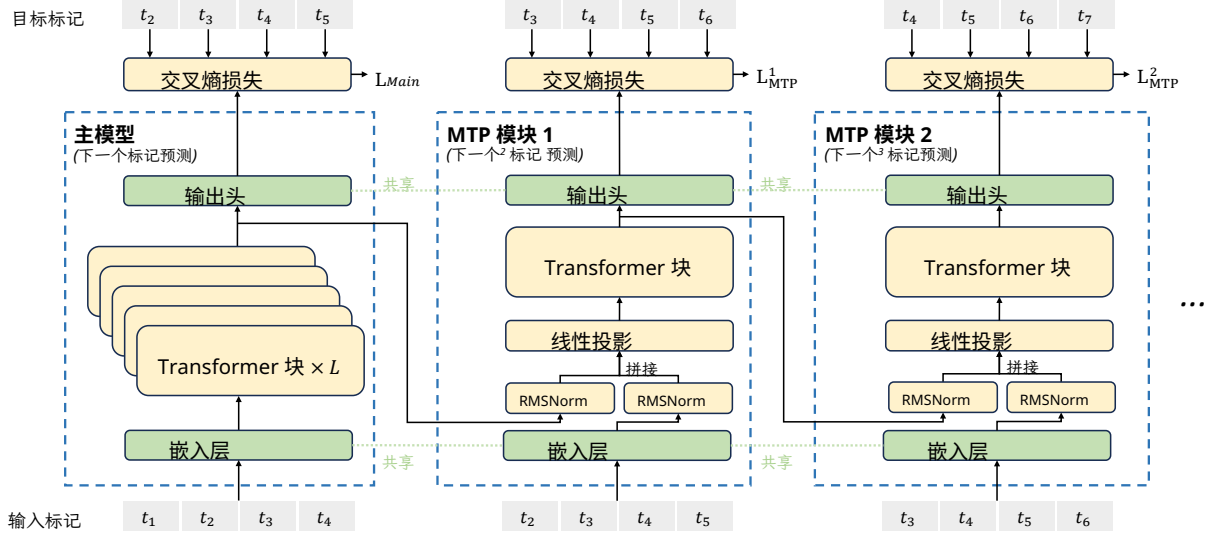


图 3 | 我们的 Multi-Token Prediction (MTP) 实现示意图。我们在每个深度为每个标记的预测保留完成的因果链。

节点受限路由。 类似 DeepSeek-V2 中使用的设备受限路由，DeepSeek-V3 也采用受限的路由机制以在训练期间限制通信开销。简而言之，我们确保每个标记将被发送至至多 M 节点，这些节点是根据每个节点上分布的专家的最高 K_M 亲和度分数之和来选择的。在该约束下，我们的 MoE 训练框架几乎可以实现完全的计算-通信重叠。

不丢弃 Token。 由于有效的负载均衡策略，DeepSeek-V3 在整个训练期间保持良好的负载均衡。因此，DeepSeek-V3 在训练过程中不会丢弃任何 Token。此外，我们还实施了特定的部署策略以确保推理负载均衡，因此 DeepSeek-V3 在推理时也不会丢弃 Token。

2.2. 多-Token 预测

受 Gloeckle 等人 (2024年) 启发，我们为 DeepSeek-V3 探索并设定了一个多Token预测 (MTP) 目标函数，将预测范围扩展到每个位置的多个未来 Token。一方面，MTP 目标函数使训练信号更为密集，并可能提升数据效率。另一方面，MTP 可能使模型能够预先规划其表示，以更好地预测未来的 Token。图3展示了我们对 MTP 的实现。不同于 Gloeckle 等人 (2024年)，其使用独立的输出头并行地预测 D 个额外的 Token，我们依次预测额外的 Token，并在每个预测深度保持因果链的完成。我们在本节介绍我们的 MTP 实现的细节。

MTP 模块。 具体而言，我们的 MTP 实现使用 D 个串行模块来预测 D 个额外的 Token。第 k 个 MTP 模块由一个共享嵌入层 $\text{Emb}(\cdot)$ 、一个共享输出头 $\text{OutHead}(\cdot)$ 、一个 Transformer 块 $\text{TRM}_k(\cdot)$ 、以及一个投影矩阵 $M_k \in \mathbb{R}^{d \times 2d}$ 组成。对于第 i 个输入标记 t_i ，在第 k 个预测深度处，我们首先将第 $(k-1)$ 个深度下第 i 个标记的表示 $\mathbf{h}_i^{k-1} \in \mathbb{R}^d$ 与第 $(i+k)$ 个标记的嵌入 $\text{Emb}(t_{i+k}) \in \mathbb{R}^d$ 合并

通过线性投影：

$$\mathbf{h}_i'^k = M_k[\text{RMSNorm}(\mathbf{h}_i^{k-1}); \text{RMSNorm}(\text{Emb}(t_{i+k}))], \quad (21)$$

其中 $[\cdot; \cdot]$ 表示连接。特别地，当 $k = 1$ 时， $\mathbf{h}_i^{k-1}$ 指主模型给出的表示。请注意，对于每个 MTP 模块，其嵌入层与主模型共享。合并后的 $\mathbf{h}_i'^k$ 作为第 k 个深度处的 Transformer 块的输入，以产生当前深度的输出表示 $\mathbf{h}_i^k$ ：

$$\mathbf{h}_{1:T-k}^k = \text{TRM}_k(\mathbf{h}_{1:T-k}'), \quad (22)$$

其中 T 表示输入序列长度， $i:j$ 表示切片操作（左右边界均包含）。最后，以 $\mathbf{h}_i^k$ 作为输入，共享的输出头将为第 k 个额外预测标记 $p_{i+1+k}^k \in \mathbb{R}^V$ 计算概率分布，其中 V 为词表大小：

$$p_{i+k+1}^k = \text{OutHead}(\mathbf{h}_i^k). \quad (23)$$

输出头 $\text{OutHead}(\cdot)$ 将表示线性映射到 logits，随后应用 $\text{Softmax}(\cdot)$ 函数来计算第 k 个额外标记的预测概率。此外，对于每个 MTP 模块，其输出头与主模型共享。我们保持预测因果链的原则与 EAGLE (Li 等, 2024年b) 相似，但其主要目标函数是推测式解码 (Leviathan 等, 2023年; Xia 等, 2023年)，而我们利用 MTP 来改进训练。

MTP 训练目标函数。 对于每个预测深度，我们计算交叉熵损失 $\mathcal{L}^k_{\text{MTP}}$ ：

$$\mathcal{L}_{\text{MTP}}^k = \text{CrossEntropy}(p_{2+k:T+1}^k, t_{2+k:T+1}) = -\frac{1}{T} \sum_{i=2+k}^{T+1} \log p_i^k[t_i], \quad (24)$$

其中 T 表示输入序列长度， t_i 表示第 i 个位置的真实标记， $p_i^k[t_i]$ 表示对 t_i 的对应预测概率，由第 k 个 MTP 模块给出。最后，我们对所有深度上的 MTP 损失取平均，并乘以权重因子 λ ，得到整体 MTP 损失 $\mathcal{L}_{\text{MTP}}$ ，其作为 DeepSeek-V3 的额外训练目标函数：

$$\mathcal{L}_{\text{MTP}} = \frac{\lambda}{D} \sum_{k=1}^D \mathcal{L}_{\text{MTP}}^k. \quad (25)$$

推理中的 MTP。 我们的 MTP 策略主要旨在提升主模型的性能，因此在推理期间，我们可以直接丢弃 MTP 模块，主模型也能独立、正常运行。此外，我们还可以将这些 MTP 模块用于投机式解码，以进一步改善生成延迟。

3. 基础设施

3.1. 计算集群

DeepSeek-V3 在配备有 2048 块 NVIDIA H800 GPU 的集群上进行训练。H800 集群中的每个节点包含 8 块 GPU，节点内通过 NVLink 和 NVSwitch 连接。跨不同节点，采用 InfiniBand (IB) 互连以促进通信。

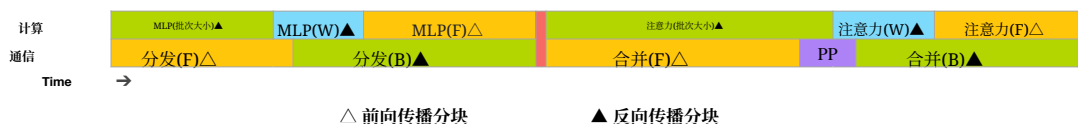


图 4 | 针对一对独立的前向传播和反向传播分块的重叠策略（Transformer 块的边界未对齐）。橙色表示前向传播，绿色表示"对输入的反向传播"，蓝色表示"对权重的反向传播"，紫色表示 PP 通信，红色表示屏障。全对全通信和 PP 通信都可以被完全隐藏。

3.2. 训练框架

DeepSeek-V3 的训练由 HAI-LLM 框架提供支持，这是我们工程师从零打造的高效、轻量级训练框架。总体而言，DeepSeek-V3 采用 16 路流水线并行（PP）（Qiet al., 2023a）、64 路专家并行（EP）（Lepikhin et al., 2021），跨越 8 个节点，并使用 ZeRO-1 数据并行（DP）（Rajb-handari et al., 2020）。

为了促进 DeepSeek-V3 的高效训练，我们进行了细致的工程优化。首先，我们设计了用于高效流水线并行的 DualPipe 算法。与现有 PP 方法相比，DualPipe 具有更少的流水线气泡。更重要的是，它在前向传播和反向传播流程之间将计算与通信阶段重叠，从而解决跨节点专家并行引入的沉重通信开销问题。其次，我们开发了高效的跨节点全对全通信内核，以充分利用 IB 和 NVLink 带宽，并节省用于通信的 Streaming Multiprocessors (SM)。最后，我们精细优化了训练过程中的内存占用，使我们无需使用代价高昂的张量并行（TP）即可训练 DeepSeek-V3。

3.2.1. DualPipe 与计算-通信重叠

对于 DeepSeek-V3，由跨节点专家并行引入的通信开销导致约为 1:1 的低效计算-通信比。为应对这一挑战，我们设计了一种创新的流水线并行算法，名为 DualPipe，它不仅通过将前向传播与反向计算的计算-通信阶段有效重叠来加速模型训练，还能够减少流水线气泡。

DualPipe 的关键思想是在一对独立的前向传播块与反向传播块内，将计算与通信进行重叠。具体而言，我们将每个块划分为四个组件：attention，all-to-all dispatch，MLP，以及 all-to-all combine。特别地，对于一个反向传播块，attention 和 MLP 都会进一步被拆分为两部分，backward for input 和 backward for weights，这与 Zero Bubble (Qi 等, 2023b) 中的做法类似。此外，我们还有一个 PP communication 组件。如下图4所示，对于一对前向传播和反向传播块，我们会重新排列这些组件，并手动调整用于通信与计算的 GPU SM（流式多处理器）比例。在这种重叠策略下，我们可以确保全对全通信和 PP 通信在执行期间都能够被完全隐藏。基于这一高效的重叠策略，完整的 DualPipe 调度如图 5 所示。它采用双向流水线调度，同时从流水线两端馈入微批次，并且相当一部分通信可以被完全重叠。这种重叠还确保：随着模型进一步缩放，只要我们保持恒定的计算与通信比例，我们仍可在跨节点使用细粒度专家，同时将全对全通信开销降至近乎为零。



图 5 | 在两个方向上，针对 8 个 PP ranks 和 20 个微批次的 DualPipe 调度示例。反向方向中的微批次与前向方向中的微批次是对称的，因此为简化说明，我们省略其批次 ID。由共享黑色边框包围的两个单元具有相互重叠的计算和通信。

方法	气泡	参数	激活
1F1B	$(PP - 1)(F + B)$	$1\times$	PP
ZB1P	$(PP - 1)(F + B - 2W)$	$1\times$	PP
DualPipe (本方法)	$(\frac{PP}{2} - 1)(F+B + B - 3W)$	$2\times$	$PP + 1$

表 2 | 不同流水线并行方法的流水线气泡和内存使用情况比较。 F 表示一个前向传播块的执行时间， B 表示一个完整反向传播块的执行时间， W 表示一个“backward for weights”块的执行时间， 以及 $F\&B$ 表示两个互相重叠的前向传播与反向传播块的执行时间。

此外，即便在没有沉重通信负担的更一般场景中，DualPipe 仍然表现出效率优势。在表2中，我们总结了不同 PP 方法下的流水线气泡和内存使用情况。正如表中所示，与 ZB1P (Qi et al., 2023b) 和 1F1B (Harlap et al., 2018年) 相比，DualPipe 显著减少了流水线气泡，同时仅将峰值激活内存增加了 $\frac{1}{PP}$ 倍。尽管 DualPipe 需要保留两份模型参数，但由于我们在训练期间使用较大的 EP 大小，这并不会显著增加内存消耗。与 Chimera (Li and Hoefler, 2021年) 相比，DualPipe 只要求流水线阶段数和微批次数能被 2 整除，而不要求微批次能被流水线阶段数整除。此外，对于 DualPipe，随着微批次数量的增加，流水线气泡和激活内存都不会增加。

3.2.2. 跨节点全对全通信的高效实现

为确保 DualPipe 拥有充足的计算性能，我们定制了高效的跨节点全对全通信内核（包括分发和合并），以节省专用于通信的 SM 数量。这些内核的实现与专家混合门控算法以及我们集群的网络拓扑协同设计。具体而言，在我们的集群中，跨节点 GPU 通过 IB 完全互连，节点内通信通过 NVLink 进行。NVLink 提供 160 GB/s 的带宽，约为 IB (50 GB/s) 的 3.2 倍。为有效利用 IB 与 NVLink 的不同带宽，我们将每个标记的分发限制在最多 4 个节点，从而减少 IB 流量。对于每个标记，当其路由决策确定后，会首先通过 IB 被传输到其目标节点上具有相同节点内索引的 GPU。一旦到达目标节点，我们会尽力确保它能通过 NVLink 立即转发到承载其目标专家的特定 GPU，且不被随后到达的标记阻塞。如此，IB 与 NVLink 的通信可实现充分重叠，每个标记在每个节点上可高效选择平均 3.2 位专家，且不会引入额外的 NVLink 开销。这意味着，尽管 DeepSeek-V3

在实践中仅选择 8 个已路由专家，它可以在保持相同通信开销的情况下，将该数量缩放至最多 13 个专家（4 个节点 $\times$ 3.2 位专家/节点）。总体而言，在这样的通信策略下，仅需 20 个 SM 便足以充分利用 IB 与 NVLink 的带宽。

具体而言，我们采用 warp 专用化技术 (Bauer et al., 2014)，并将 20 个 SM 划分为 10 条通信通道。在分发过程中，(1) IB 发送，(2) IB 到 NVLink 的转发，(3) NVLink 接收，分别由各自的 warp（束）处理。为每个通信任务分配的 warp（束）数量会根据所有 SM 上的实际负载动态调整。类似地，在合并过程中，(1) NVLink 发送，(2) NVLink 到 IB 的转发与累加，(3) IB 接收与累加，也由动态调整的 warp（束）处理。此外，分发与合并内核都与计算流重叠，因此我们也考虑它们对其他 SM 计算内核的影响。具体而言，我们使用定制的 PTX (Parallel Thread Execution) 指令，并对通信分块大小进行自动调优，从而显著减少对 L2 缓存的占用以及对其他 SM 的干扰。

3.2.3. 极致节省内存，开销极小

为了在训练期间降低内存占用，我们采用以下技术

s.

重新计算 RMSNorm 和 MLA 上投影。 我们在反向传播期间重新计算所有的 RMSNorm 操作和 MLA 上投影，从而无需持久存储它们的输出激活。仅带来很小的开销，这一策略显著降低用于存储激活的内存需求。

在 CPU 上的指数移动平均。 在训练期间，我们保留模型参数的指数移动平均 (EMA)，用于在学习率衰减后对模型性能进行早期估计。EMA 参数存储在 CPU 内存中，并在每个训练步数之后异步更新。该方法使我们能够在不产生额外内存或时间开销的情况下维持 EMA 参数。

用于多标记预测的共享嵌入与输出头。 借助 DualPipe 策略，我们将模型最浅的层（包括嵌入层）和最深的层（包括输出头）部署在同一 PP rank 上。该安排使得在 MTP 模块与主模型之间，共享嵌入与输出头的参数和梯度可以实现物理共享。此类物理共享机制进一步提升我们的内存效率。

3.3. FP8 训练

受低精度训练的最新进展启发 (Dettmers 等, 2022年; Noune 等, 2022年; Peng 等, 2023年b)，我们提出了一种用于训练 DeepSeek-V3 的、采用 FP8 数据格式的细粒度混合精度框架。尽管低精度训练前景广阔，但激活、权重和梯度中的离群值常常构成限制 (Fishman 等, 2024年; He 等; Sun 等, 2024年)。尽管推理量化方面已取得显著进展 (Frantar 等, 2022年; Xiao 等, 2023年)，但在大规模语言模型中成功应用低精度技术的研究仍相对较少

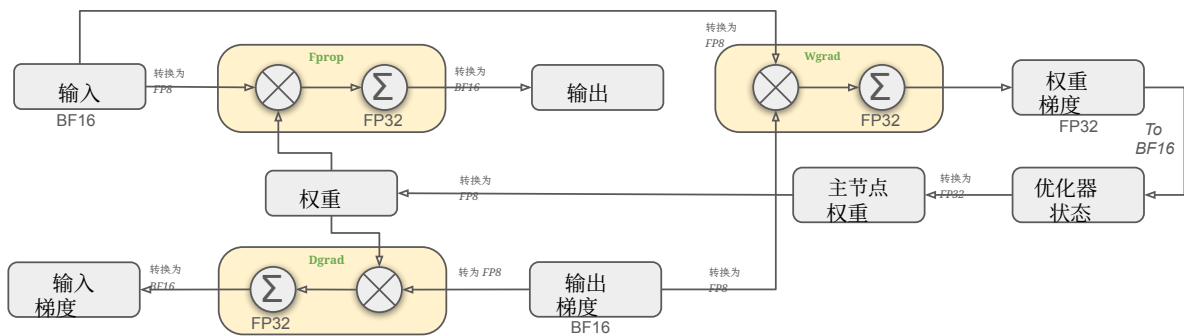


图 6 | 采用 FP8 数据格式的整体混合精度框架。为便于说明，仅展示 Linear 算子。

预训练（Fishman 等，2024年）。为应对这一挑战并有效扩展 FP8 格式的动态范围，我们提出了一种细粒度量化策略：按瓦片分组（每组 $1 \times N_c$ 个元素）或按块分组（每组 $N_c \times N_c$ 个元素）。在我们更高精度的累加过程中，相关的反量化开销得到大幅缓解，这对实现准确的 FP8 GEMM（通用矩阵乘法）至关重要。此外，为进一步降低 MoE 训练中的内存与通信开销，我们以 FP8 缓存并分发激活，同时以 BF16 存储低精度的优化器状态。我们在两个模型规模上验证了所提出的 FP8 混合精度框架，这两个规模与 DeepSeek-V2-Lite 和 DeepSeek-V2 相近，训练约 1 万亿个标记（详见附录 B.1）。值得注意的是，与 BF16 基线相比，我们的 FP8 训练模型的相对损失误差始终低于 0.25%，远在训练随机性可接受范围之内。

3.3.1. 混合精度框架

基于在低精度训练中被广泛采用的技术（Kalamkar 等，2019年；Narang 等，2017年），我们提出了一种用于 FP8 训练的混合精度框架。在该框架中，大多数计算密集型操作在 FP8 中进行，而少数关键操作则策略性地保持其原始数据格式，以平衡训练效率与数值稳定性。整体框架如图 6 所示。

首先，为了加速模型训练，大多数核心计算内核，即 GEMM 运算，以 FP8 精度实现。这些 GEMM 运算以 FP8 张量为输入，并产生 BF16 或 FP32 的输出。如图 6 所示，与 Linear 算子相关的三种 GEMM 运算，即 Fprop（前向传播）、Dgrad（激活的反向传播）和 Wgrad（权重的反向传播），均在 FP8 中执行。与原始的 BF16 方法相比，该设计在理论上将计算速度提高一倍。此外，FP8 Wgrad GEMM 允许将激活以 FP8 存储，以用于反向传播。这显著降低了内存消耗。

尽管 FP8 格式在效率上具有优势，某些算子由于对低精度计算较为敏感，仍然需要更高的精度。此外，一些低成本算子也可以在对整体训练成本几乎不带来开销的情况下使用更高精度。因此，经过仔细研究，我们对以下组件保持原始精度（例如，BF16 或 FP32）：嵌入模块、输出头、专家混合门控模块、归一化算子以及注意算子。这些针对性的高精度保留确保了 DeepSeek-V3 的训练稳定性。为进一步保证数值稳定性，我们以更高精度存储主权重、权重梯度和优化器状态。

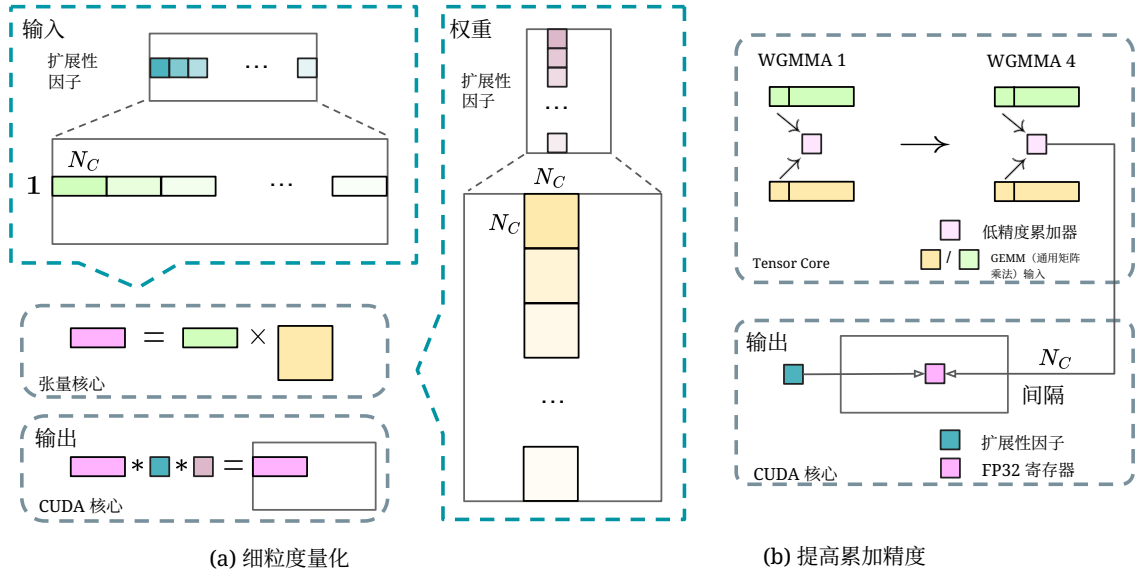


图 7 | (a) 我们提出了一种细粒度量化的方法，以减轻由特征离群值引起的量化误差；为便于说明，仅展示 Fprop。 (b) 结合我们的量化策略，我们通过以 $N_C = 128$ 个元素为间隔将 MMA 提升到 CUDA 核心以进行高精度累加，从而提升 FP8 GEMM（通用矩阵乘法）精度。

尽管这些高精度组件会带来一些内存开销，但通过在我们的分布式训练系统中跨多个 DP ranks 进行高效分片，可以将它们的影响降到最低。

3.3.2. 通过量化与乘法提升精度

基于我们的混合精度 FP8 框架，我们提出若干策略以提升低精度训练准确率，既关注量化方法，也关注乘法过程。

细粒度量化。 在低精度训练框架中，由于 FP8 格式的指数位减少、动态范围受限，溢出和下溢是常见挑战。作为一种常见做法，通过将输入张量的最大绝对值缩放到 FP8 的最大可表示值，使输入分布对齐到 FP8 格式的可表示范围（Narang 等人，2017 年）。这种方法使低精度训练对激活的离群值高度敏感，从而会严重降低量化准确率。为此，我们提出了一种细粒度的量化方法，在更细的粒度上进行缩放。如图 7 (a) 所示，(1) 对激活，按 1×128 瓦片为单位对元素分组并进行缩放（即，每个标记、每 128 个通道）；以及 (2) 对权重，按 128×128 块为单位对元素分组并进行缩放（即，每 128 个输入通道与每 128 个输出通道）。这种方法通过针对更小的元素组自适应地调整缩放，使量化过程更好地适应离群值。在附录 B.2 中，我们进一步讨论了当像权重量化一样按块粒度对激活进行分组与缩放时的训练不稳定性。

我们方法的一个关键改动是在 GEMM 运算的内维度上引入逐组缩放因子。标准的 FP8 GEMM 并不直接支持这一功能。然而，结合我们精确的 FP32 累加策略，它可以

被高效地实现。

值得注意的是，我们的细粒度量化策略与微缩放格式的理念高度一致（Rouhani et al., 2023b），而英伟达下一代 GPU（Blackwell 系列）的 Tensor Cores 已宣布支持量化粒度更小的微缩放格式（英伟达, 2024a）。我们希望我们的设计能够为未来工作提供参考，以跟上最新的 GPU 架构。

提高累加精度。 低精度 GEMM 运算常常遭受下溢问题，其准确率在很大程度上依赖于高精度的累加，这通常以 FP32 精度执行（Kalamkar et al., 2019年; Narang et al., 2017年）。然而，我们观察到，在英伟达 H800 GPU 上，FP8 GEMM 的累加精度仅能保留约14位，显著低于 FP32 的累加精度。当内层维度 K 很大时（Wortsman et al., 2023年），这一问题会更加突出；这在大规模模型训练中很常见，因为此时批大小和模型宽度都会增大。以两个具有 $K = 4096$ 的随机矩阵的 GEMM 运算为例，在我们的初步测试中，Tensor Cores 中受限的累加精度会导致最大相对误差接近2%。尽管存在这些问题，受限的累加精度在少数 FP8 框架中仍是默认选项（英伟达, 2024b），这严重限制了训练准确率。

为了解决这一问题，我们采用提升到 CUDA Cores 以获得更高精度的策略（Thakkar 等, 2023年）。该过程如图 7（b）所示。具体而言，在 Tensor Cores 上执行 MMA（矩阵乘加）时，中间表示结果使用受限的比特宽度进行累加。一旦间隔为 N_C 时，这些部分结果将被拷贝到 CUDA Cores 上的 FP32 寄存器，在那里进行全精度 FP32 累加。如前所述，我们的细粒度量化沿内部维度 K 按组应用缩放因子。这些缩放因子可在 CUDA Cores 上高效地作为反量化过程进行乘法运算，且额外的计算成本最小。

值得注意的是，此修改会降低单个线程束组的 WGMMA（线程束组级矩阵乘加）指令发射率。然而，在 H800 架构上，通常会有两个 WGMMA 并发驻留：当一个线程束组执行提升操作时，另一个能够执行 MMA 操作。这种设计使两种操作能够重叠，从而保持 Tensor Cores 的高利用率。根据我们的实验，将 $N_C = 128$ 个元素（相当于 4 个 WGMMA）设为在不引入大量开销的情况下显著提升精度的最小累加间隔。

尾数优先于指数。 与先前工作采用的混合 FP8 格式形成对比（英伟达, 2024年b; Peng 等, 2023年b; Sun 等, 2019年b），该格式在 Fprop 中使用 E4M3（4位指数和3位尾数），在 Dgrad 和 Wgrad 中使用 E5M2（5位指数和2位尾数），我们在所有张量上采用 E4M3 格式以获得更高的精度。我们将这种方法的可行性归功于我们的细粒度量化策略，即瓦片级和块级缩放。通过在更小的元素组上进行操作，我们的方法在这些成组元素之间有效共享指数位，从而缓解有限动态范围的影响。

在线量化。 延迟量化被用于按张量的量化框架（英伟达, 2024b; Peng 等, 2023b），其维护了最大绝对

值在先前迭代中的历史，以推断当前数值。为确保精确的缩放并简化框架，我们为每个 1×128 激活瓦片或 128×128 权重块在线计算最大绝对值。在此基础上，我们推导出缩放因子，然后将激活或权重在线量化为 FP8 格式。

3.3.3. 低精度存储与通信

结合我们的 FP8 训练框架，我们通过将缓存的激活和优化器状态压缩为更低精度的格式，进一步降低内存消耗和通信开销。

低精度优化器状态。 我们采用 BF16 数据格式替代 FP32，在 AdamW (Loshchilov 和 Hutter, 2017年) 优化器中跟踪一阶和二阶矩，而未出现可观察的性能下降。然而，为了在整个训练过程中保证数值稳定性，主权重（由优化器存储）和梯度（用于批大小累积）仍保留为 FP32。

低精度激活。 如图 6 所示，Wgrad 操作以 FP8 执行。为降低内存消耗，一个自然的选择是将激活以 FP8 格式缓存，用于 Linear 算子的反向传播。然而，为实现低成本的高精度训练，我们对若干算子作了特殊考虑：

(1) **关于 Linear 在注意算子之后的输入。** 这些激活也会在注意算子的反向传播中使用，因此对精度较为敏感。我们为这些激活专门采用定制的 E5M6 数据格式。此外，在反向传播中，这些激活将从 1×128 量化瓦片转换为 128×1 瓦片。为避免引入额外的量化误差，所有缩放因子都采用取整缩放，即 2 的整数次幂。

(2) **专家混合中 SwiGLU 算子的输入。** 为进一步降低内存开销，我们缓存 SwiGLU 算子的输入，并在反向传播中重新计算其输出。这些激活也采用我们的细粒度量化方法以 FP8 存储，在内存效率与计算准确率之间取得平衡。

低精度通信。 通信带宽是 MoE 模型训练中的关键瓶颈。为缓解这一挑战，我们将 MoE 上投影之前的激活量化为 FP8，然后应用 dispatchcomponents，该方法与 MoE 上投影中的 FP8 Fprop 兼容。类似于注意算子之后的 Linear 的输入，此处激活的缩放因子为 2 的整数次幂。对 MoE 下投影之前的激活梯度也采用类似策略。对于前向传播和反向传播的 combine components，我们将其保持为 BF16，以在训练流水线的关键部分保留训练精度。

3.4. 推理与部署

我们在 H800 集群上部署 DeepSeek-V3，其中每个节点内的 GPU 通过 NVLink 互连，集群内的所有 GPU 通过 IB 完全互连。为同时确保在线服务的服务级别目标 (SLO) 与高吞吐量，我们采用如下部署策略，将预填充与解码阶段分离。

3.4.1. 预填充

预填充阶段的最小部署单元由 4 个节点（共 32 块 GPU）构成。该 attention 部分采用 4 路张量并行（TP4）和序列并行（SP），并结合 8 路数据并行（DP8）。其较小的 TP 规模为 4，限制了 TP 通信的开销。对于 MoE 部分，我们采用 32 路专家并行（EP32），以确保每个专家处理足够大的批次大小，从而提升计算效率。对于 MoE 全对全通信，我们沿用训练中的相同方法：先通过 IB 跨节点传输 Token，再通过 NVLink 在节点内的 GPU 之间转发。尤其地，为节省 TP 通信，我们在浅层的稠密 MLP 中使用 1 路张量并行。

为了在该 MoE 部分实现不同专家之间的负载均衡，我们需要确保每块 GPU 处理的标记数量大致相同。为此，我们引入冗余专家的部署策略，即复制高负载专家并进行冗余部署。高负载专家根据在线部署期间收集的统计数据进行检测，并定期调整（例如，每 10 分钟）。在确定冗余专家集合后，我们会根据观测到的负载，在同一节点内的 GPU 之间仔细地重新安排专家，力求在不增加跨节点全对全通信开销的情况下尽可能平衡各 GPU 的负载。在部署 DeepSeek-V3 时，我们为预填充阶段设置了 32 个冗余专家。对于每块 GPU，除了其原本承载的 8 个专家外，还将额外承载 1 个冗余专家。

此外，在预填充阶段，为了提高吞吐量并隐藏全对全通信和 TP 通信的开销，我们同时处理两组计算工作负载相近的微批次，将一个微批次的 attention 和 MoE 与另一个微批次的 dispatch 和 combine 重叠。

最后，我们正在探索一种针对专家的动态冗余策略，其中每个 GPU 承载更多专家（例如 16 个专家），但在每个推理步骤中仅会激活 9 个。在每一层的全对全通信开始之前，我们会即时计算全局最优的路由方案。鉴于预填充阶段涉及大量计算，计算该路由方案的开销几乎可以忽略不计。

3.4.2. 解码

在解码过程中，我们将共享专家视为已路由专家。从这个角度看，每个标记在路由时会选择 9 个专家，其中共享专家被视为重负载专家，将始终被选择。解码阶段的最小部署单元由 40 个节点和 320 块 GPU 组成。attention 部分采用 TP4 与 SP，结合 DP80，而 MoE 部分使用 EP320。对于 MoE 部分，每个 GPU 仅承载 1 个专家，另有 64 块 GPU 负责承载冗余专家和共享专家。dispatch 和 combine 部分的全对全通信通过 IB 的直接点对点传输来实现，以达到低延迟。此外，我们利用 IBGDA（英伟达，2022年）技术，进一步降低延迟并提升通信效率。

与预填充类似，我们基于在线服务中的专家负载统计，周期性地在一定时间间隔内确定冗余专家集合。不过，由于每个 GPU 仅承载一个专家，我们无需重新排列专家。我们还在探索用于解码的动态冗余策略。但这需要更细致地优化用于计算全局最优路由方案的算法，并与 dispatch 内核进行融合，以降低开销。

此外，为提升吞吐量并隐藏全对全通信的开销，我们也在探索在解码阶段同时处理两段计算工作量相近的微批次。不同于预填充，`attention` 在解码阶段占用更大比例的时间。因此，我们将一个微批次的 `attention` 与另一个微批次的 `dispatch+MoE+combine` 进行重叠。在解码阶段，每个专家的批大小较小（通常在 256 Token 以内），瓶颈在于内存访问而非计算。由于 MoE 部分只需加载一个专家的参数，内存访问开销极小，因此使用较少的 SM 不会显著影响整体性能。因此，为避免影响 `attention` 部分的计算速度，我们可以仅为 `dispatch+MoE+combine` 分配少量 SM。

3.5. 硬件设计建议

基于我们对全对全通信与 FP8 训练方案的实现，我们向 AI 硬件厂商提出以下芯片设计建议。

3.5.1. 通信硬件

在 DeepSeek-V3 中，我们实现了计算与通信的重叠，以在计算过程中隐藏通信延迟。与串行的计算与通信相比，这显著降低了对通信带宽的依赖。然而，当前的通信实现依赖于昂贵的 SM 资源（例如，我们在 H800 GPU 上将可用的 132 个 SM 中的 20 个用于该目的），这会限制计算吞吐量。此外，用 SM 执行通信会带来显著低效，因为 `Tensor Cores` 几乎完全未被利用。

目前，SM 主要为全对全通信执行以下任务：

- **转发数据** 在 IB (InfiniBand) 与 NVLink 域之间，同时在单张 GPU 上聚合同一节点内面向多个 GPU 的 IB 流量。
- **传输数据** 在 RDMA 缓冲区（已注册的 GPU 内存区域）与输入/输出缓冲区之间。
- **执行 reduce 操作** 用于 `all-to-all combine`。
- **管理细粒度内存布局** 在将分块数据传输到跨 IB 和 NVLink 域的多个专家期间。

我们期望未来的厂商开发出能够将这些通信任务从宝贵的计算单元 SM（流式多处理器）上卸载的硬件，作为 GPU 协处理器或网络协处理器，例如英伟达 SHARP Graham 等（2016 年）。此外，为了降低应用程序编程复杂度，我们希望这类硬件能从计算单元的视角统一 IB（横向扩展）和 NVLink（纵向扩展）网络。通过这一统一接口，计算单元可以通过基于简单原语提交通信请求，轻松在整个 IB-NVLink 统一域内完成诸如 `read`、`write`、`multicast` 和 `reduce` 等操作。

3.5.2. 计算硬件

在 `Tensor Cores` 中提高 FP8 GEMM（通用矩阵乘法）累加精度。在当前英伟达 Hopper 架构的 `Tensor Core` 实现中，FP8 GEMM（通用矩阵乘法）的累加精度受限。根据最大指数通过右移对齐 32 个尾数乘积之后，`Tensor Core` 仅使用每个尾数乘积的最高 14 位进行加法，

并截断超出该范围的位。将加法结果累加到寄存器也仅采用 14 位精度。我们的实现通过在 CUDA 核中以 FP32 精度将 128 次 FP8×FP8 乘法的加法结果累加到寄存器，部分缓解了这一限制。尽管这有助于成功开展 FP8 训练，但由于 Hopper 架构在 FP8 GEMM（通用矩阵乘法）累加精度方面的硬件缺陷，它只是权衡性的折中方案。未来的芯片需要采用更高的精度。

支持瓦片级与块级量化。当前的 GPU 仅支持按张量量化，缺乏对诸如我们提出的瓦片级与块级这类细粒度量化的原生支持。在当前实现中，当达到 N_C 间隔时，会将部分和从 Tensor Cores 复制到 CUDA cores，与缩放因子相乘后，加到 CUDA cores 上的 FP32 寄存器中。尽管结合我们精确的 FP32 累加策略已显著缓解反量化开销，但 Tensor Cores 与 CUDA cores 之间频繁的数据移动仍然限制了计算效率。因此，我们建议未来芯片通过使 Tensor Cores 能够接收缩放因子并以分组缩放执行 MMA，来支持细粒度量化。这样，整个部分和的累加与反量化都可以一直在 Tensor Cores 内直接完成，直到产生最终结果，从而避免频繁的数据移动。

支持在线量化。尽管我们的研究已证明其有效性，但当前实现仍难以有效支持在线量化。在现有流程中，我们需要从 HBM（高带宽内存）读取 128 个 BF16 激活值（上一轮计算的输出）进行量化；量化得到的 FP8 值随后又写回 HBM，之后为执行 MMA 再次被读取。为解决这一低效问题，我们建议未来芯片将 FP8 转换与对 TMA（Tensor Memory Accelerator）的访问融合为单个操作，使量化能在激活从全局内存传输到共享内存的过程中完成，从而避免频繁的内存读写。我们还建议支持线程束级转换指令以提升加速比，这将进一步促进层归一化与 FP8 转换的更好融合。或者，可采用近内存计算的方法，将计算逻辑放置在 HBM 附近。在这种情况下，BF16 元素在从 HBM 读入 GPU 的同时即可直接转换为 FP8，将片外内存访问约减少 50%。

支持转置的 GEMM 运算。当前架构使将矩阵转置与 GEMM 运算进行融合变得繁琐。在我们的工作流中，前向传播期间的激活会被量化为 1×128 FP8 瓦片并存储。在反向传播期间，需要将该矩阵读出、反量化、转置、重新量化为 128×1 瓦片，并存入 HBM。为减少内存操作，我们建议未来的芯片在 MMA 操作之前，针对在训练和推理中都需要的精度，从共享内存直接以转置方式读取矩阵。配合将 FP8 格式转换与 TMA 访问进行融合，这一改进将显著简化量化工作流。

4. 预训练

4.1. 数据构建

与 DeepSeek-V2 相比，我们通过提高数学与编程样本的比例来优化预训练语料，同时将多语言覆盖范围扩展到超出

英语和中文。此外，我们优化了数据处理流水线，在保持语料多样性的同时尽量减少冗余。受 Ding 等人（2024年）启发，我们为保证数据完整性实现了文档打包方法，但在训练过程中未引入跨样本注意掩蔽。最后，DeepSeek-V3 的训练语料在我们的分词器下包含 14.8T 的高质量且多样的 Token。

在 DeepSeekCoder-V2（DeepSeek-AI, 2024年a）的训练过程中，我们观察到 Fill-in-Middle（FIM）策略在使模型能够基于上下文线索准确预测中间文本的同时，并不会削弱下一标记预测能力。与 DeepSeekCoder-V2 保持一致，我们也在 DeepSeek-V3 的预训练中引入了 FIM 策略。具体而言，我们采用 Prefix-Suffix-Middle（PSM）框架将数据构造如下形式：

`<|fim_begin|>f前缀<|fim_hole|>f后缀<|fim_end|>f中间<|eos_token|>`

该结构在文档级别应用，作为预打包流程的一部分。FIM 策略以 0.1 的比例应用，与 PSM 框架保持一致。

DeepSeek-V3 的分词器采用 Byte-level BPE（Shibata et al., 1999），并配备 128K Token 的扩展词表。为了优化多语言压缩效率，我们对分词器的预分词器和训练数据进行了修改。此外，与 DeepSeek-V2 相比，新的预分词器引入了将标点与换行合并的 Token。然而，当模型处理没有末尾换行的多行提示时，尤其是用于少样本评测的提示，这一技巧可能会引入标记边界偏差（Lundberg, 2023年）。为了解决这一问题，我们在训练过程中随机拆分一定比例的此类合并 Token，使模型接触到更广泛的特殊情况，从而缓解该偏差。

4.2. 超参数

模型超参数。我们将 Transformer 架构的层数设为 61 层，隐藏维度设为 7168。所有可学习参数均以标准差 0.006 随机初始化。在 MLA 中，我们将注意力头的数量 n_h 设为 128，将每头维度 d_h 设为 128。KV 压缩维度 d_c 设为 512，查询压缩维度 d_v 设为 1536。对于解耦的查询和键，我们将每头维度 d^R_h 设为 64。除前 3 层外，我们用 MoE 层替换所有前馈网络。每个 MoE 层由 1 个共享专家和 256 个已路由专家组成，其中每个专家的中间隐藏维度为 2048。在已路由专家中，每个标记将激活 8 个专家，并确保每个标记最多发送到 4 个节点。多标记预测深度 D 设为 1，即除了精确的下一个标记外，每个标记还将额外预测一个标记。与 DeepSeek-V2 一样，DeepSeek-V3 也在压缩后的潜在向量之后引入额外的 RMSNorm 层，并在宽度瓶颈处乘以额外的缩放因子。在该配置下，DeepSeek-V3 共包含 6710 亿参数，其中每个标记激活 370 亿。

训练超参数。我们采用 AdamW 优化器（Loshchilov and Hutter, 2017年），其超参数设为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.95$ ，以及权重衰减 = 0.1。在预训练期间，我们将最大序列长度设置为 4K，并在 14.8T 个标记上对 DeepSeek-V3 进行预训练。关于学习率调度，我们首先在最初的 2K 步中将其从 0 线性增加到 2.2×10^{-4} 。随后，我们保持 2.2×10^{-4} 的恒定学习率，直到模型消耗 10T 个训练标记。之后，我们在 4.3T 个标记内将学习率逐步衰减至 2.2×10^{-5} ，遵循余弦衰减曲线。在最后的 500B 个标记的训练中，我们在最初的 333B 个标记内保持 2.2×10^{-5} 的恒定学习率，并切换到另一种恒定学习率

为 7.3×10^{-6} ，在剩余的 167B 个标记内使用。梯度裁剪范数设为 1.0。我们采用批大小调度策略：在前 469B 个标记的训练中，批大小从 3072 逐步增至 15360，随后在剩余训练中保持 15360。我们利用流水线并行，将模型的不同层部署在不同的 GPU 上；对于每一层，已路由的专家将均匀部署在属于 8 个节点的 64 张 GPU 上。对于节点受限路由，每个标记最多会被发送到 4 个节点（即， $M = 4$ ）。对于无辅助损失的负载均衡，在前 14.3T 个标记中，我们将偏置更新速度 γ 设为 0.001，在剩余的 500B 个标记中设为 0.0。对于平衡损失，我们将 α 设为 0.0001，仅用于避免任一单个序列内出现极端失衡。MTP 损失权重 λ 在前 10T 个标记中设为 0.3，在剩余的 4.8T 个标记中设为 0.1。

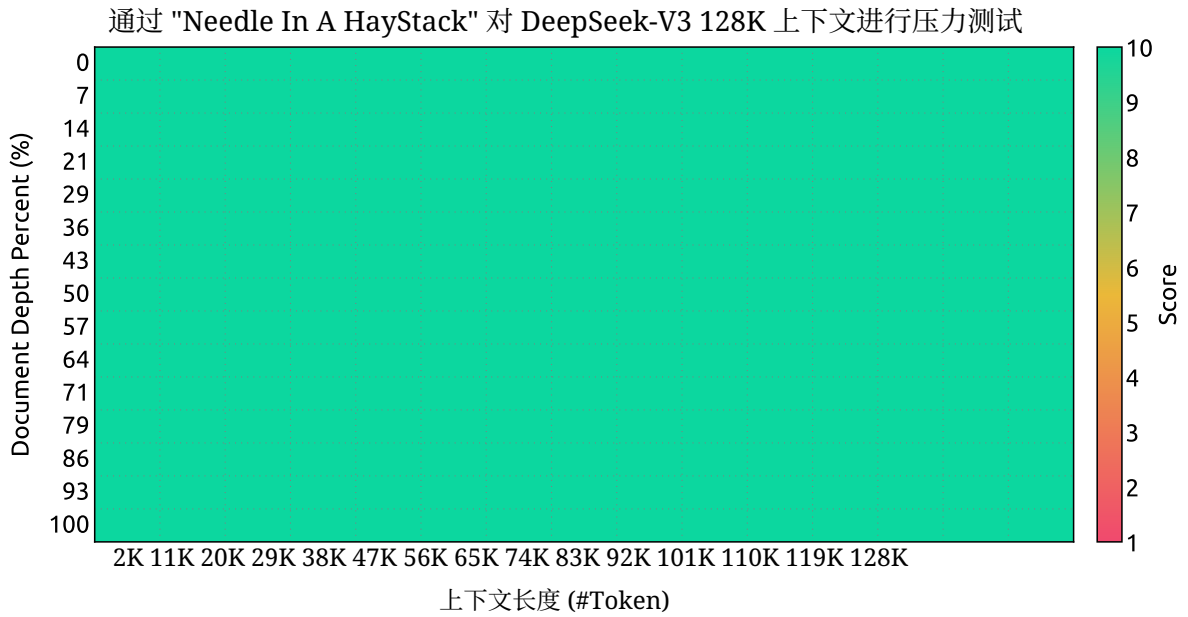


图 8 | 在 "Needle In A Haystack" (NIAH) 测试上的评估结果。DeepSeek-V3 在所有上下文窗口长度（最长至 128K）上表现良好。

4.3. 长上下文扩展

我们采用与 DeepSeek-V2 (DeepSeek-AI, 2024年c) 类似的方法，为 DeepSeek-V3 提供长上下文能力。在预训练阶段之后，我们采用 YaRN (Peng 等, 2023年a) 进行上下文扩展，并额外进行两个训练阶段，每个阶段包含 1000 步数，逐步将上下文窗口从 4K 扩展到 32K，再到 128K。YaRN 的配置与 DeepSeek-V2 中使用的一致，仅应用于解耦的共享键 $\mathbf{k}_t^R$ 。超参数在两个阶段中保持一致，缩放 $s = 40$ 、 $\alpha = 1$ 、 $\beta = 32$ ，以及缩放因子 $\sqrt{t} = 0.1 \ln s + 1$ 。在第一阶段，序列长度设为 32K，批大小为 1920。第二阶段将序列长度提高到 128K，并将批大小降低至 480。两个阶段的学习率均设为 7.3×10^{-6} ，与预训练阶段的最终学习率一致。

通过这种两阶段的扩展训练，DeepSeek-V3 在保持强劲性能的同时，能够处理长度最高达 128K 的输入。图 8 表明，DeepSeek-V3 在监督微调之后，在 "Needle In A Haystack" (NIAH) 测试上取得了显著的性能，展示出在最长达 128K 的上下文窗口长度上的一致稳健性。

4.4. 评估

4.4.1. 评估基准

DeepSeek-V3 的基础模型在一个多语言语料库上进行了预训练，其中英语和中文占多数，因此我们在一系列主要为英语和中文的基准上评估其性能，同时也在一个多语言基准上进行评估。我们的评测基于集成在我们 HAI-LLM 框架中的内部评估框架。所考虑的基准按如下类别列出，其中带下划线的基准为中文，带双下划线的基准为多语言：

多学科多项选择 数据集包括 MMLU (Hendrycks 等人, 2020年)、MMLU-Redux (Gema 等人, 2024年)、MMLU-Pro (Wang 等人, 2024b)、MMMLU (OpenAI, 2024b)、C-Eval (Huang 等人, 2023年) 以及 CMMLU (Li 等人, 2023年)。

语言理解与推理 数据集包括 HellaSwag (Zellers 等人, 2019年)、PIQA (Bisk 等人, 2020年)、ARC (Clark 等人, 2018年) 以及 BigBench Hard (BBH) (Suzgun 等人, 2022年)。

闭卷问答 数据集包括 TriviaQA (Joshi 等人, 2017年) 和 NaturalQuestions (Kwiatkowski 等人, 2019年)。

阅读理解 数据集包括 RACE (Lai et al., 2017年)、DROP (Dua et al., 2019年)、C3 (Sun et al., 2019a) 和 CMRC (Cui et al., 2019年)。

指代消解 数据集包括 CLUEWSC (Xu et al., 2020年) 和 WinoGrande (Sakaguchi et al., 2019年)。

语言建模 数据集包括 Pile (Gao et al., 2020年)。

中文理解与文化 数据集包括 CCPM (Li et al., 2021年)。

数学 数据集包括 GSM8K (Cobbe et al., 2021年)、MATH (Hendrycks et al., 2021年)、MGSM (Shi et al., 2023年) 和 CMATH (Wei et al., 2023年)。

代码 数据集包括 HumanEval (Chen et al., 2021年)、LiveCodeBench-Base (0801-1101) (Jain et al., 2024年)、MBPP (Austin et al., 2021年) 和 CRUXEval (Gu et al., 2024年)。

标准化考试 包括 AGIEval (Zhong 等, 2023年)。注意，AGIEval 包含英文和中文子集。

延续我们以往的工作 (DeepSeek-AI, 2024b,c)，我们对包括 HellaSwag、PIQA、WinoGrande、RACE-Middle、RACE-High、MMLU、MMLU-Redux、MMLU-Pro、MMMLU、ARC-Easy、ARC-Challenge、C-Eval、CMMLU、C3 和 CCPM 在内的数据集采用基于困惑度的评估，对 TriviaQA、NaturalQuestions、DROP、MATH、GSM8K、MGSM、HumanEval、MBPP、LiveCodeBench-Base、CRUXEval、BBH、AGIEval、CLUEWSC、CMRC 和 CMATH 采用基于生成的评估。此外，我们对 Pile-test 执行基于语言建模的评估，并使用 Bits-Per-Byte (BPB) 作为指标，以确保使用不同分词器的模型之间的公平比较。

4.4.2. 评估结果

在表3中，我们将 DeepSeek-V3 的基础模型与当前最先进水平的开源基础模型进行对比，包括 DeepSeek-V2-Base (DeepSeek-AI, 2024c) (我们之前的发布)、Qwen2.5 72B Base (Qwen, 2024b) 以及 LLaMA-3.1 405B Base (AI@Meta, 2024b)。我们使用内部评估框架对所有这些模型进行评测，并确保它们采用相同的评估设置。请注意，由于过去几个月我们的评估框架发生了变化，性能

基准 (指标)		# 示例数	DeepSeek-V2 Base	Qwen2.5 LLaMA-3.1 72B 基础版 405B 基础版		DeepSeek-V3 Base
架构	-		MoE	稠密	稠密	MoE
# 激活参数	-		21B	72B	405B	37B
# 总参数	-		236B	72B	405B	671B
英语	Pile-test (BPB)	-	0.606	0.638	0.542	0.548
	BBH (EM)	3-shot	78.8	79.8	82.9	87.5
	MMLU (EM)	5-shot	78.4	85.0	84.4	87.1
	MMLU-Redux (EM)	5 次示例	75.6	83.2	81.3	86.2
	MMLU-Pro (EM)	5 次示例	51.4	58.3	52.8	64.4
	DROP (F1)	3 次示例	80.4	80.6	86.0	89.0
	ARC-Easy (EM)	25-shot	97.6	98.4	98.4	98.9
	ARC-Challenge (EM)	25-shot	92.2	94.5	95.3	95.3
	HellaSwag (EM)	10-shot	87.1	84.8	89.2	88.9
	PIQA (EM)	0-shot	83.9	82.6	85.9	84.7
	Winogrande 数据集 (EM)	5-shot	86.3	82.3	85.2	84.9
	RACE-Middle (EM)	5-shot	73.1	68.1	74.2	67.1
	RACE-High (EM)	5-shot	52.6	50.3	56.8	51.3
	TriviaQA (EM)	5-shot	80.0	71.9	82.7	82.9
	NaturalQuestions (EM)	5-shot	38.6	33.2	41.5	40.0
	AGIEval (EM)	0-shot	57.5	75.8	60.6	79.6
Code	HumanEval (Pass@1)	0-shot	43.3	53.0	54.9	65.2
	MBPP (Pass@1)	3-shot	65.0	72.6	68.4	75.4
	LiveCodeBench-Base (Pass@1)	3-shot	11.6	12.9	15.5	19.4
	CRUXEval-I (EM)	2-shot	52.5	59.1	58.5	67.3
	CRUXEval-O (EM)	2-shot	49.8	59.9	59.9	69.8
Math	GSM8K (EM)	8-shot	81.6	88.3	83.5	89.3
	MATH (EM)	4-shot	43.4	54.4	49.0	61.6
	MGSM (EM)	8-shot	63.6	76.2	69.9	79.8
	CMath (EM)	3-shot	78.7	84.5	77.3	90.7
中文	CLUEWSC (EM)	5-shot	82.0	82.5	83.0	82.7
	C-Eval (EM)	5-shot	81.4	89.2	72.5	90.1
	CMMLU (EM)	5-shot	84.0	89.5	73.7	88.8
	CMRC (EM)	1-shot	77.4	75.8	76.0	76.3
	C3 (EM)	零样本	77.4	76.7	79.7	78.6
	CCPM (EM)	零样本	93.0	88.5	78.6	92.0
多语言	MMMLU-non-English (EM)	五样本	64.0	74.8	73.8	79.4

表 3 | DeepSeek-V3-Base 与其他具有代表性的开源基础模型的比较。所有模型均在我们的内部框架中评测，并采用相同的评测设置。得分差距不超过 0.3 视为同一水平。DeepSeek-V3-Base 在大多数基准上取得了最佳性能，尤其在数学和代码任务上。

DeepSeek-V2-Base 的性能与我们之前报告的结果相比存在轻微差异。总体而言，DeepSeek-V3-Base 全面优于 DeepSeek-V2-Base 和 Qwen2.5 72B Base，并且在大多数基准上超过 LLaMA-3.1 405B Base，基本上已成为最强的开源模型。

从更细致的角度看，我们分别将 DeepSeek-V3-Base 与其他开源基础模型进行对比。（1）与 DeepSeek-V2-Base 相比，得益于我们在模型架构上的改进、模型规模与训练 Token 的扩增以及数据质量的提升，DeepSeek-V3-Base 如预期地取得了显著更好的性能。（2）与当前最先进水平的中文开源模型 Qwen2.5 72B Base 相比，DeepSeek-V3-Base 仅用一半的被激活参数，也展现出显著优势，

尤其在英文、多语言、代码和数学基准上。对于中文基准，除 CMMLU（中文多学科多项选择任务）外，DeepSeek-V3-Base 的性能也优于 Qwen2.5 72B。（3）与 LLaMA-3.1 405B Base（其被激活参数数量是 DeepSeek-V3-Base 的 11 倍的最大开源模型）相比，DeepSeek-V3-Base 在多语言、代码和数学基准上也表现更好得多。对于英文和中文语言基准，DeepSeek-V3-Base 表现具有竞争力或更优，且在 BBH、MMLU-series、DROP、C-Eval、CMMLU 和 CCPM 上尤为出色。

得益于我们高效的架构和全面的工程优化，DeepSeek-V3 实现了极高的训练效率。在我们的训练框架和基础设施下，DeepSeek-V3 每训练 1 万亿 Token 仅需 18 万 H800 GPU 小时，其成本远低于训练 72B 或 405B 稠密模型。

基准（指标）	# 示例数	小型专家混合 基线	小型专家混合 使用 MTP	大型专家混合 基线	大型专家混合 使用 MTP
# 已激活参数 (Inference)	-	2.4B	2.4B	209亿	209亿
# 参数总量 (Inference)	-	157亿	157亿	2287亿	2287亿
# 训练 Token	-	1.33万 亿	1.33万 亿	540B	540B
Pile-test (BPB)	-	0.729	0.729	0.658	0.657
BBH (EM)	3-shot	39.0	41.4	70.0	70.7
MMLU (EM)	5-shot	50.0	53.3	67.5	66.6
DROP (F1)	1-shot	39.2	41.3	68.5	70.6
TriviaQA (EM)	5-shot	56.9	57.7	67.0	67.3
NaturalQuestions (EM)	5-shot	22.7	22.3	27.2	28.5
HumanEval (Pass@1)	0-shot	20.7	26.8	44.5	53.7
MBPP (Pass@1)	3-shot	35.8	36.8	61.6	62.2
GSM8K (EM)	8-shot	25.4	31.4	72.3	74.0
MATH (EM)	4-shot	10.7	12.6	38.6	39.8

表 4 | MTP 策略的消融结果。MTP 策略在大多数评估基准上稳定提升模型性能。

4.5. 讨论

4.5.1. 多标记预测的消融研究

在表4中，我们展示了 MTP 策略的消融结果。具体而言，我们在不同规模的两个基线模型之上验证了 MTP 策略。小规模下，我们在 1.33 万亿个标记上训练了一个总参数数量为 157 亿的基线专家混合模型。大规模下，我们在 5400 亿个标记上训练了一个总参数数量为 2287 亿的基线专家混合模型。在此基础上，在保持训练数据和其他架构不变的情况下，我们在它们上附加一个深度为 1 的 MTP 模块，并使用 MTP 策略再训练两个模型以进行比较。需要注意的是，在推理期间，我们直接丢弃 MTP 模块，因此被比较模型的推理成本完全相同。从该表可以观察到，MTP 策略在大多数评测基准上都能持续提升模型性能。

4.5.2. 无辅助损失的负载均衡策略的消融研究

在表5中，我们展示了无辅助损失的负载均衡策略的消融结果。我们在两个不同缩放的基线模型之上对该策略进行了验证。在小缩放下，我们在 1.33T Token 上训练了一个包含 15.7B 个总参数的基线专家混合模型。在大缩放下，我们在 578B Token 上训练了一个包含 228.7B 个总参数的基线专家混合模型。

基准（指标）	# 次数	小型专家混合	小型专家混合	大型专家混合	大型专家混合
		基于辅助损失	无辅助损失	基于辅助损失	无辅助损失
# 已激活参数	-	2.4B	2.4B	20.9B	209亿
# 参数总量	-	157亿	157亿	2287亿	2287亿
# 训练 Token	-	1.33T	1.33T	578B	578B
Pile-test (BPB)	-	0.727	0.724	0.656	0.652
BBH (EM)	3-shot	37.3	39.3	66.7	67.9
MMLU (EM)	5-shot	51.0	51.8	68.3	67.2
DROP (F1)	1-shot	38.1	39.0	67.1	67.1
TriviaQA (EM)	5-shot	58.3	58.5	66.7	67.7
NaturalQuestions (EM)	5-shot	23.2	23.4	27.1	28.1
HumanEval (Pass@1)	0-shot	22.0	22.6	40.2	46.3
MBPP (Pass@1)	3-shot	36.6	35.8	59.2	61.2
GSM8K (EM)	8-shot	27.1	29.6	70.7	74.5
MATH (EM)	4-shot	10.9	11.1	37.2	39.6

表 5 | 无辅助损失均衡策略的消融结果。与纯粹基于辅助损失的方法相比，无辅助损失策略在大多数评估基准上持续取得更好的模型性能。

这两个基线模型都纯粹依赖辅助损失来促进负载均衡，并采用带有 Top-k 亲和度归一化的 Sigmoid 门控函数。用于控制辅助损失强度的超参数分别与 DeepSeek-V2-Lite 和 DeepSeek-V2 保持一致。在这两个基线模型之上，在保持训练数据和其他架构不变的前提下，我们移除了所有辅助损失，并引入了无辅助损失的负载均衡策略以作对比。从表中可以观察到，无辅助损失的策略在大多数评估基准上都能持续取得更好的模型性能。

4.5.3. 按批次的负载均衡 VS. 按序列的负载均衡

无辅助损失的均衡与序列级辅助损失之间的关键区别在于其均衡范围：按批次与按序列。与序列级辅助损失相比，批次级均衡施加的约束更为灵活，因为它不会对每个序列强制实现序列内的均衡。这种灵活性使专家能够在不同领域上更好地专精化。为验证这一点，我们记录并分析了在 Pile 测试集的不同领域上，一个 16B 基于辅助损失的基线模型和一个 16B 无辅助损失模型的专家负载。如图9所示，我们观察到，无辅助损失的模型如预期展现出更强的专家专精模式。

为进一步探究这种灵活性与模型性能优势之间的相关性，我们另外设计并验证了一种批次级辅助损失，它鼓励在每个训练批次上而非每个序列上实现负载均衡。实验结果表明，当达到相近的批次级负载均衡水平时，批次级辅助损失也可以获得与无辅助损失方法相近的模型性能。具体而言，在我们使用 1B MoE 模型的实验中，验证损失分别为：2.258（使用序列级辅助损失）、2.253（使用无辅助损失方法）以及 2.253（使用批次级辅助损失）。我们在 3B MoE 模型上也观察到类似结果：使用序列级辅助损失的模型的验证损失为 2.085，而使用无辅助损失方法或批次级辅助损失的模型的验证损失均为 2.080。

此外，尽管批次级负载均衡方法表现出一致的性能优势，但它们在效率方面也面临两个潜在挑战：（1）内部的负载不均衡

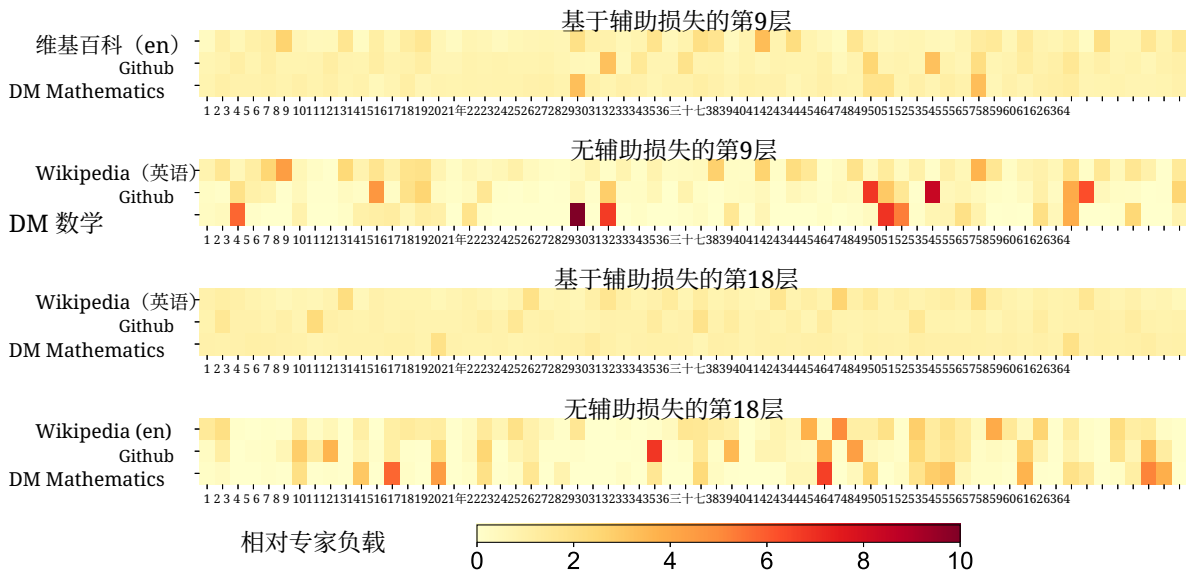


图 9 | 无辅助损失与基于辅助损失的模型在 Pile 测试集三个领域上的专家负载。无辅助损失的模型相比基于辅助损失的模型呈现出更强的专家专门化模式。相对专家负载表示实际专家负载与理论均衡专家负载之间的比值。由于篇幅限制，我们仅以两层的结果为例展示，所有层的结果见附录 C。

出现在某些序列或小批次中，以及（2）推理期间由领域偏移引起的负载不均衡。第一个挑战由我们的训练框架自然解决：该框架采用大规模专家并行和数据并行，保证每个微批次的规模较大。针对第二个挑战，我们也设计并实现了一个高效的推理框架，采用冗余专家部署，如第 3.4 节所述，以加以克服。

5. 后训练

5.1. 有监督微调

我们精心整理用于指令微调的数据集，覆盖多个领域，共计150万条实例；每个领域均采用针对其特定需求定制的不同数据生成方法。

推理数据。 对于与推理相关的数据集（包括侧重数学、代码竞赛问题和逻辑谜题的数据集），我们使用内部的 DeepSeek-R1 模型生成数据。具体而言，尽管 R1 生成的数据在准确率方面表现出色，但也存在过度思考、格式不佳以及篇幅过长等问题。我们的目标是兼顾 R1 生成推理数据的高准确率与常规格式推理数据的清晰与简洁。

为建立我们的方法，我们首先使用合并的有监督微调（SFT）与强化学习（RL）训练流水线，开发一个面向特定领域的专家模型，例如 代码、数学 或 通用推理。该专家模型作为最终模型的数据生成器。训练过程针对每个实例生成两种不同类型的 SFT 样本：第一种将问题与其原始回答配对，格式为 <问题，原始回答>，而第二种则加入系统提示词

连同问题和 R1 回答一起，格式为 <系统提示词，问题，R1 回答>。

系统提示词经过精心设计，包含指令，以引导模型生成融入反思与验证机制的回答。在 RL 阶段，即便没有显式的系统提示词，模型也利用高温采样生成回答，融合来自 R1 生成数据与原始数据的模式。经过数百个 RL 步数之后，处于中间表示阶段的 RL 模型学会了纳入 R1 模式，从而有策略地提升整体性能。

在完成 RL 训练阶段后，我们实施拒绝采样，为最终模型策集高质量的 SFT 数据，其中将专家模型用作数据生成源。该方法确保最终训练数据保留 DeepSeek-R1 的优势，同时生成简洁且有效的回答。

非推理数据。对于创意写作、角色扮演和简单问答等非推理数据，我们使用 DeepSeek-V2.5 生成回答，并邀请人工标注者核验数据的准确率与正确性。

SFT 设置。我们使用 SFT 数据集对 DeepSeek-V3-Base 进行两轮微调，采用余弦衰减学习率调度，从 5×10^{-6} 启动，并逐渐降低到 1×10^{-6} 。在训练期间，每个单独的序列由多个样本打包而成。但我们采用样本掩蔽策略，以确保这些示例彼此隔离、互不可见。

5.2. 强化学习

5.2.1. 奖励模型

We 在我们的强化学习（RL）过程中，采用基于规则的奖励模型（RM）和基于模型的 RM。

基于规则的 RM。对于可通过特定规则验证的问题，我们采用基于规则的奖励机制来确定反馈。例如，某些数学问题具有确定性的结果，我们要求模型以指定格式（例如，用方框标出）给出最终答案，从而便于我们依据规则验证其正确性。类似地，对于 LeetCode 问题，我们可以利用编译器根据测试用例生成反馈。在可能的情况下采用基于规则的验证，可确保更高的可靠性，因为这种方法不易被操纵或利用。

基于模型的 RM。对于具有自由形式真实答案的问题，我们依赖奖励模型来判断回答是否与预期真实答案相匹配。相反，对于没有明确真实答案的问题（例如创意写作类问题），奖励模型的任务是基于问题及相应答案作为输入提供反馈。该奖励模型基于 DeepSeek-V3 的 SFT 检查点进行训练。为提升其可靠性，我们构建偏好数据，不仅提供最终奖励，还包含通向该奖励的思维链。该方法有助于缓解在特定任务中的奖励黑客风险。

5.2.2. 组相对策略优化

类似于 DeepSeek-V2 (DeepSeek-AI, 2024c), 我们采用 Group Relative Policy Optimization (GRPO) (Shao et al., 2024年), 该方法省去了通常与策略模型规模相当的 critic 模型, 转而依据组内得分来估计基线。具体而言, 对于每个问题 q , GRPO 从旧策略模型 $\pi_{\theta_{old}}$ 中采样一组输出 $\{o_1, o_2, \dots, o_G\}$, 随后通过最大化如下目标函数来优化策略模型 π_{θ} :

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (26)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (27)$$

其中, ϵ 和 β 是超参数; π_{ref} 是参考模型; A_i 是优势项, 由每个组内输出所对应的奖励 $\{r_1, r_2, \dots, r_G\}$ 计算得到:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (28)$$

我们在 RL 过程中纳入来自多种领域的提示, 例如编码、数学、写作、角色扮演和问答。这种方法不仅使模型更贴近人类偏好, 还提升了在基准上的性能, 尤其是在可用的 SFT 数据有限的场景中。

5.3. 评估

5.3.1. 评估设置

评估基准。 除了用于基础模型测试的基准之外, 我们进一步在 IFEval (Zhou et al., 2023年)、FRAMES (Krishna et al., 2024年)、LongBench v2 (Bai et al., 2024年)、GPQA (Rein et al., 2023年)、SimpleQA (OpenAI, 2024c)、C-SimpleQA (He et al., 2024年)、SWE-Bench Verified (OpenAI, 2024d)、Aider 1、LiveCodeBench (Jain et al., 2024年) (问题来自2024年8月至2024年11月)、Codeforces 2、Chinese National High School Mathematics Olympiad (CNMO 2024年)³, 以及 American Invitational Mathematics Examination 2024年 (AIME 2024年) (MAA, 2024年) 上对指令模型进行评估。

对比基线。 我们对我们的对话模型进行了全面评测, 并与多种强劲的基线方法进行对比, 包括 DeepSeek-V2-0506、DeepSeek-V2.5-0905、Qwen2.5 72B Instruct、LLaMA-3.1 405B Instruct、Claude-Sonnet-3.5-1022 和 GPT-4o-0513。对于 DeepSeek-V2 模型系列, 我们选择了最具代表性的变体进行对比。对于闭源模型, 我们通过其各自的 API 进行评测。

详细评测配置。 对于包括 MMLU、DROP、GPQA 和 SimpleQA 在内的标准基准, 我们采用 simple-evals 框架⁴提供的评估提示。

¹<https://aider.chat>

²<https://codeforces.com>

³<https://www.cms.org.cn/Home/comp/comp/cid/12.html>

⁴<https://github.com/openai/simple-evals>

在零样本设置下，我们对 MMLU-Redux 使用 Zero-Eval 提示格式（Lin，2024年）。对于其他数据集，我们遵循其原始评估协议，使用数据集创作者提供的默认提示。对于代码和数学基准，HumanEval-Mul 数据集共包含 8 种主流编程语言（Python、Java、Cpp、C#、JavaScript、TypeScript、PHP 和 Bash）。我们使用 CoT 和非 CoT 方法评估模型在 LiveCodeBench 上的性能，其数据收集时间为 2024年8月到2024年11月。Codeforces 数据集使用参赛者百分比进行度量。SWE-Bench verified 使用无代理框架进行评估（Xia 等人，2024年）。我们使用“diff”格式评估与 Aider 相关的基准。在数学评测方面，AIME 和 CNMO 2024 采用温度 0.7 进行评估，结果取 16 次运行的平均值；而 MATH-500 使用贪心解码。我们允许所有模型在每个基准上最多输出 8192 个标记。

基准（指标）		DeepSeek V2-0506	DeepSeek V2.5-0905	Qwen2.5 72B-Inst.	LLaMA-3.1 405B-Inst.	Claude-3.5-Sonnet-1022	GPT-4o 0513	DeepSeek V3
英语	架构	MoE	MoE	稠密	稠密	-	-	MoE
	# 已激活参数	21B	21B	72B	405B	-	-	37B
	# 总参数量	236B	236B	72B	405B	-	-	671B
	MMLU (EM)	78.2	80.6	85.3	88.6	88.3	87.2	88.5
	MMLU-Redux (EM)	77.9	80.3	85.6	86.2	88.9	88.0	89.1
	MMLU-Pro (EM)	58.5	66.2	71.6	73.3	78.0	72.6	75.9
	DROP (3-shot F1)	83.0	87.8	76.7	88.7	88.3	83.7	91.6
	IF-Eval (Prompt Strict)	57.7	80.6	84.1	86.0	86.5	84.3	86.1
	GPQA-Diamond (Pass@1)	35.3	41.3	49.0	51.1	65.0	49.9	59.1
Code	SimpleQA (正确)	9.0	10.2	9.1	17.1	28.4	38.2	24.9
	FRAMES (准确率)	66.9	65.4	69.8	70.0	72.5	80.5	73.3
	LongBench v2 (Acc.)	31.6	35.4	39.4	36.1	41.0	48.1	48.7
	HumanEval-Mul (Pass@1)	69.3	77.4	77.3	77.2	81.7	80.5	82.6
	LiveCodeBench (Pass@1-COT)	18.8	29.2	31.1	28.4	36.3	33.4	40.5
	LiveCodeBench (Pass@1)	20.3	28.4	28.7	30.1	32.8	34.2	37.6
	Codeforces (百分位)	17.5	35.6	24.8	25.3	20.3	23.6	51.6
Math	SWEVerified (已解决)	-	22.6	23.8	24.5	50.8	38.8	42.0
	Aider-Edit (Acc.)	60.3	71.6	65.4	63.9	84.2	72.9	79.7
	Aider-Polyglot (Acc.)	-	18.2	7.6	5.8	45.3	16.0	49.6
	AIME 2024 (Pass@1)	4.6	16.7	23.3	23.3	16.0	9.3	39.2
	MATH-500 (EM)	56.3	74.7	80.0	73.8	78.3	74.6	90.2
中文	CNMO 2024 (Pass@1)	2.8	10.8	15.9	6.8	13.1	10.8	43.2
	CLUEWSC (EM)	89.9	90.4	91.4	84.7	85.4	87.9	90.9
	C-Eval (EM)	78.6	79.5	86.1	61.5	76.7	76.0	86.5
	C-SimpleQA (正确率)	48.5	54.1	48.4	50.4	51.3	59.3	64.8

表 6 | DeepSeek-V3 与其他具有代表性的聊天模型的比较。所有模型都在将输出长度限制为 8K 的配置下进行评估。包含少于 1000 个样本的基准会在不同温度设置下多次测试，以得到稳健的最终结果。DeepSeek-V3 是性能最好的开源模型，同时在与前沿闭源模型的对比中也表现出具有竞争力的性能。

5.3.2. 标准评测

表6给出了评测结果，显示 DeepSeek-V3 是性能最优的开源模型。此外，它与前沿的闭源模型（如 GPT-4o 和 Claude-3.5-Sonnet）相比也具有竞争力。

英语基准。 MMLU 是一个广泛认可的基准，用于评估跨不同知识领域和任务的大型语言模型的性能。DeepSeek-V3 展现出具有竞争力的性能，与 LLaMA-3.1-405B、GPT-4o 和 Claude-Sonnet 3.5 等顶尖模型不相上下，同时显著优于 Qwen2.5 72B。此外，在更具挑战性的教育知识基准 MMLU-Pro 上，DeepSeek-V3 表现出色，紧随 Claude-Sonnet 3.5 之后。在 MMLU-Redux（一个修正标签后的 MMLU 精炼版本）上，DeepSeek-V3 超越了其同侪。此外，在博士级评测测试集 GPQA-Diamond 上，DeepSeek-V3 取得了出色的成绩，名列仅次于 Claude 3.5 Sonnet，并以较大优势领先于其他所有竞争者。

在诸如 DROP、LongBench v2 和 FRAMES 等长上下文理解基准上，DeepSeek-V3 继续展现其顶级模型地位。在 DROP 的 3-shot 设置下，其取得了令人印象深刻的 91.6 F1 分数，超越该类别的所有其他模型。在 FRAMES 上（该基准要求在 100k 标记上下文上进行问答），DeepSeek-V3 紧随 GPT-4o 之后，同时以显著优势领先其他所有模型。这表明 DeepSeek-V3 在处理极长上下文任务方面的强大能力。DeepSeek-V3 的长上下文能力还通过其在 LongBench v2 上的同类最佳性能得到进一步验证，该数据集在 DeepSeek V3 发布前仅几周才发布。在事实知识基准 SimpleQA 上，DeepSeek-V3 落后于 GPT-4o 和 Claude-Sonnet，主要由于其设计侧重与资源分配。DeepSeek-V3 将更多训练标记用于学习中文知识，因此在 C-SimpleQA 上表现卓越。在指令遵循基准上，DeepSeek-V3 相较其前代 DeepSeek-V2-series 有显著提升，突显其在理解并遵循用户定义格式约束方面的能力增强。

代码与数学基准。 对于 LLMs 而言，编程是一项具有挑战性且实用的任务，涵盖如 SWE-Bench-Verified 和 Aider 等工程导向的任务，以及 HumanEval 和 LiveCodeBench 等算法任务。在工程任务中，DeepSeek-V3 虽不及 Claude-Sonnet-3.5-1022，但相较开源模型具有显著优势。开源的 DeepSeek-V3 有望推进与编程相关的工程任务的发展。通过提供对其强大能力的访问，DeepSeek-V3 能够推动软件工程与算法开发等领域的创新与改进，帮助开发者和研究人员拓展开源模型在编程任务中所能达到的边界。在算法任务中，DeepSeek-V3 展现出更优的性能，在 HumanEval-Mul 和 LiveCodeBench 等基准上超越所有基线方法。这一成功可归因于其先进的知识蒸馏技术，该技术有效提升了其在算法导向任务中的代码生成与问题求解能力。

在数学基准上，DeepSeek-V3 展现出卓越的性能，显著超越基线方法，并为非 o1-like 模型设定了新的当前最先进水平。具体而言，在 AIME、MATH-500 和 CNMO 2024 年上，DeepSeek-V3 在绝对分数上较第二佳的模型 Qwen2.5 72B 领先约 10%，对于如此具有挑战性的基准而言，这是一个相当可观的优势。这一卓越能力凸显了来自 DeepSeek-R1 的蒸馏技术的有效性，事实证明这对非 o1-like 模型非常有益。

中文基准。 Qwen 和 DeepSeek 是两个对中文和英语均有良好支持的具有代表性的模型系列。在事实性基准 Chinese SimpleQA 上，DeepSeek-V3 以 16.4 分的优势超过了 Qwen2.5-72B，尽管 Qwen2.5 的训练语料规模更大，包含 18T 个标记，比 DeepSeek-V3 的 14.8T 个标记多 20%

模型	Arena-Hard	AlpacaEval 2.0
DeepSeek-V2.5-0905	76.2	50.5
Qwen2.5-72B-Instruct	81.2	49.1
LLaMA-3.1 405B	69.3	40.5
GPT-4o-0513	80.4	51.1
Claude-Sonnet-3.5-1022	85.2	52.0
DeepSeek-V3	85.5	70.0

表 7 | 英文开放式对话评测。对于 AlpacaEval 2.0，我们使用长度受控的胜率作为指标。

预训练于。

在 C-Eval（一个用于评估中文教育知识的代表性基准）和 CLUEWSC (Chinese Winograd Schema Challenge) 上，DeepSeek-V3 与 Qwen2.5-72B 表现出相近的性能水平，这表明这两个模型都已针对具有挑战性的中文推理与教育类任务进行了良好优化。

5.3.3. 开放式评测

除了标准基准之外，我们还使用 LLMs 作为评审对开放式生成任务进行评测，结果如表7所示。具体而言，我们遵循 AlpacaEval 2.0（Dubois 等, 2024年）和 Arena-Hard（Li 等, 2024a）的原始配置，这两者均使用 GPT-4-Turbo-1106 作为评审进行成对比较。在 Arena-Hard 上，DeepSeek-V3 相比于基线 GPT-4-0314 取得了超过 86% 的亮眼胜率，表现与 Claude-Sonnet-3.5-1022 等顶尖模型不相上下。这凸显了 DeepSeek-V3 的强大能力，尤其是在处理复杂提示方面，包括编码与调试等任务。此外，DeepSeek-V3 成为首个在 Arena-Hard 基准上突破 85% 的开源模型，达成了具有里程碑意义的突破。这一成就显著缩小了开源与闭源模型之间的性能差距，为开源模型在挑战性领域所能达到的水平树立了新的标杆。

同样地，DeepSeek-V3 在 AlpacaEval 2.0 上展现出卓越的性能，超越了闭源与开源模型。这表明其在写作类任务和处理简单问答场景方面具备出色能力。值得注意的是，它以20%的显著优势超过 DeepSeek-V2.5-0905，凸显了在应对简单任务上的大幅改进，并展示了其改进措施的有效性。

5.3.4. DeepSeek-V3 作为生成式 *RewardModel*

我们将 DeepSeek-V3 的判断能力与当前最先进水平的模型进行对比，即 GPT-4o 和 Claude-3.5。表8 展示了这些模型在 RewardBench（Lambert et al., 2024年）中的性能。DeepSeek-V3 的性能与 GPT-4o-0806 和 Claude-3.5-Sonnet-1022 的最佳版本不相上下，同时超过了其他版本。此外，DeepSeek-V3 的判断能力也可以通过投票技术得到提升。因此，我们采用 DeepSeek-V3 结合投票，对开放式问题提供自我反馈，从而提升对齐过程的有效性和鲁棒性。

模型	对话	对话-困难	安全	推理	平均
GPT-4o-0513	96.6	70.4	86.7	84.9	84.7
GPT-4o-0806	96.1	76.1	88.1	86.6	86.7
GPT-4o-1120	95.8	71.3	86.2	85.2	84.6
Claude-3.5-sonnet-0620	96.4	74.0	81.6	84.7	84.2
Claude-3.5-sonnet-1022	96.4	79.7	91.1	87.6	88.7
DeepSeek-V3	96.9	79.8	87.0	84.3	87.0
DeepSeek-V3 (maj@6)	96.9	82.6	89.5	89.2	89.6

Table 8 | GPT-4o、Claude-3.5-sonnet 和 DeepSeek-V3 在 RewardBenc 上的性能 h.

模型	LiveCodeBench-CoT		MATH-500	
	Pass@1	长度	Pass@1	长度
DeepSeek-V2.5 基线	31.1	718	74.6	769
DeepSeek-V2.5 +R1 蒸馏	37.4	783	83.2	1510

表 9 | 来自 DeepSeek-R1 的蒸馏贡献。Live- CodeBench 和 MATH-500 的评测设置与表6中的相同。

5.4. 讨论

5.4.1. 来自 *DeepSeek-R1* 的蒸馏

我们基于 DeepSeek-V2.5，对来自 DeepSeek-R1 的蒸馏贡献进行消融。基线在短 CoT 数据上训练，而其对比模型使用由上述专家检查点生成的数据。

表9展示了蒸馏数据的有效性，在 LiveCodeBench 和 MATH-500 基准上均取得了显著提升。我们的实验揭示了一个有趣的权衡：蒸馏带来更好的性能，但也显著增加了平均响应长度。为在模型准确率与计算效率之间保持平衡，我们在蒸馏中为 DeepSeek-V3 谨慎选择了最优设置。

我们的研究表明，从推理模型进行知识蒸馏为后训练优化提供了一条很有前景的方向。尽管我们当前的工作聚焦于从数学和代码领域蒸馏数据，这一方法在更广泛的各类任务领域中也显示出应用潜力。这些特定领域中展现出的有效性表明，long-CoT 蒸馏可能同样有助于提升在其他需要复杂推理的认知任务中的模型性能。在不同领域进一步探索这种方法，仍是未来研究的重要方向。

5.4.2. 自奖励

奖励在强化学习（RL）中发挥关键作用，引导优化过程。在可以通过外部工具轻松验证的领域，例如某些编程或数学场景，RL 表现出卓越的效果。然而，在更通用的场景中，靠硬编码构建反馈机制并不现实。在 DeepSeek-V3 的开发过程中，对于这些更广泛的情境，我们采用宪法式 AI 方法（Bai 等，2022年），将 DeepSeek-V3 自身的投票评估结果作为反馈来源。这种方法已

产生了显著的对齐效果，显著提升了 DeepSeek-V3 在主观评测中的性能。通过引入额外的宪法式输入，DeepSeek-V3 能够朝宪法式方向进行优化。我们认为，这种将补充信息与 LLMs 作为反馈来源相结合的范式至关重要。大型语言模型充当通用的处理器，能够将来自多样场景的非结构化信息转化为奖励，最终促进 LLMs 的自我改进。除了自我奖励之外，我们也致力于发掘其他通用且可扩展的奖励方法，以在通用场景中持续提升模型能力。

5.4.3. 多标记预测评估

与仅预测下一个单个标记不同，DeepSeek-V3 通过 MTP 技术预测接下来的 2 个标记。结合 speculative decoding 框架（Leviathan 等, 2023年；Xia 等, 2023年），它能够显著加速模型的解码速度。一个自然的问题是额外预测的标记的接受率。基于我们的评估，在各种生成主题下，第二个标记预测的接受率在 85% 到 90% 之间，表现出一致的可靠性。较高的接受率使 DeepSeek-V3 的解码速度显著提升，达到 1.8 倍的 TPS（Tokens Per Second）。

6. 结论、局限性与未来方向

本文介绍了 DeepSeek-V3，这是一种大型专家混合语言模型，具有总参数量 6710 亿、激活参数量 370 亿，并在 14.8T 个标记上完成训练。除 MLA 和 DeepSeekMoE 架构外，它还首次提出了用于负载均衡的无辅助损失策略，并设定了多标记预测训练目标函数，以获得更强的性能。得益于 FP8 训练的支持和精细的工程优化，DeepSeek-V3 的训练具有很高的性价比。后训练也成功地从 DeepSeek-R1 系列模型中蒸馏出推理能力。全面评估表明，DeepSeek-V3 已成为当前最强的开源模型，并达到了可与 GPT-4o 和 Claude-3.5-Sonnet 等领先闭源模型相媲美的性能。尽管性能强大，它也保持了经济的训练成本。其完整训练（包括预训练、上下文长度扩展和后训练）仅需 2.788M H800 GPU 小时。

尽管我们认可其强大的性能和高性价比，我们也意识到 DeepSeek-V3 仍存在一些局限性，尤其是在部署方面。首先，为确保高效的推理，DeepSeek-V3 的推荐部署单元相对较大，可能会给小型团队带来负担。其次，尽管我们的 DeepSeek-V3 部署策略已经实现了超过 DeepSeek-V2 两倍的端到端生成速度，仍有进一步提升的空间。值得庆幸的是，随着更先进硬件的发展，这些局限性有望被自然解决。

DeepSeek 一贯坚持开源模型的长期主义路线，旨在稳步迈向 AGI（通用人工智能）的终极目标。未来，我们计划在以下方向上战略性投入研究。

- 我们将持续研究并打磨我们的模型架构，旨在进一步提升训练与推理效率，力求逐步接近对无限上下文长度的高效支持。此外，我们将尝试突破 Transformer 架构的局限性，从而拓展其建模能力的边界。

- We will continuously iterate on the quantity and quality of our training data, and explore the incorporation of additional training signal sources, aiming to drive data scaling across a more comprehensive range of dimensions.
- We will consistently explore and iterate on the deep thinking capabilities of our models, aiming to enhance their intelligence and problem-solving abilities by expanding their reasoning length and depth.
- We will explore more comprehensive and multi-dimensional model evaluation methods to prevent the tendency towards optimizing a fixed set of benchmarks during research, which may create a misleading impression of the model capabilities and affect our foundational assessment.

References

- AI@Meta. Llama 3 model card, 2024a. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- AI@Meta. Llama 3.1 model card, 2024b. URL https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md.
- Anthropic. Claude 3.5 sonnet, 2024. URL <https://www.anthropic.com/news/claude-3-5-sonnet>.
- J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. [arXiv preprint arXiv:2108.07732](https://arxiv.org/abs/2108.07732), 2021.
- Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. [arXiv preprint arXiv:2212.08073](https://arxiv.org/abs/2212.08073), 2022.
- Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. [arXiv preprint arXiv:2412.15204](https://arxiv.org/abs/2412.15204), 2024.
- M. Bauer, S. Treichler, and A. Aiken. Singe: leveraging warp specialization for high performance on GPUs. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '14*, page 119–130, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450326568. doi: 10.1145/2555243.2555258. URL <https://doi.org/10.1145/2555243.2555258>.
- Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. PIQA: reasoning about physical commonsense in natural language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239. URL <https://doi.org/10.1609/aaai.v34i05.6239>.
- M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse,

- A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. *CoRR*, abs/1803.05457, 2018. URL <http://arxiv.org/abs/1803.05457>.
- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Y. Cui, T. Liu, W. Che, L. Xiao, Z. Chen, W. Ma, S. Wang, and G. Hu. A span-extraction dataset for Chinese machine reading comprehension. In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5883–5889, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1600. URL <https://aclanthology.org/D19-1600>.
- D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *CoRR*, abs/2401.06066, 2024. URL <https://doi.org/10.48550/arXiv.2401.06066>.
- DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *CoRR*, abs/2406.11931, 2024a. URL <https://doi.org/10.48550/arXiv.2406.11931>.
- DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. *CoRR*, abs/2401.02954, 2024b. URL <https://doi.org/10.48550/arXiv.2401.02954>.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *CoRR*, abs/2405.04434, 2024c. URL <https://doi.org/10.48550/arXiv.2405.04434>.
- T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.
- H. Ding, Z. Wang, G. Paolini, V. Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. *arXiv preprint arXiv:2404.10830*, 2024.
- D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL <https://doi.org/10.18653/v1/n19-1246>.
- Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto. Length-controlled alpacaEval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024.

- W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *CoRR*, abs/2101.03961, 2021. URL <https://arxiv.org/abs/2101.03961>.
- M. Fishman, B. Chmiel, R. Banner, and D. Soudry. Scaling FP8 training to trillion-token llms. *arXiv preprint arXiv:2409.12517*, 2024.
- E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.
- A. P. Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P. Minervini. Are we done with mmlu? *CoRR*, abs/2406.04127, 2024. URL <https://doi.org/10.48550/arXiv.2406.04127>.
- F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=pEWAcEjiU2>.
- Google. Our next-generation model: Gemini 1.5, 2024. URL <https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024>.
- R. L. Graham, D. Bureddy, P. Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenberg, M. Dubman, S. Kotchubievsky, V. Koushnir, et al. Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction. In *2016 First International Workshop on Communication Optimizations in HPC (COMHPC)*, pages 1–10. IEEE, 2016.
- A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution, 2024.
- D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang. Deepseek-coder: When the large language model meets programming - the rise of code intelligence. *CoRR*, abs/2401.14196, 2024. URL <https://doi.org/10.48550/arXiv.2401.14196>.
- A. Harlap, D. Narayanan, A. Phanishayee, V. Seshadri, N. Devanur, G. Ganger, and P. Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training, 2018. URL <https://arxiv.org/abs/1806.03377>.
- B. He, L. Noci, D. Paliotta, I. Schlag, and T. Hofmann. Understanding and minimising outlier features in transformer training. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. *arXiv preprint arXiv:2411.07140*, 2024.
- D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.
- Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.
- N. Jain, K. Han, A. Gu, W. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. CoRR, abs/2403.07974, 2024. URL <https://doi.org/10.48550/arXiv.2403.07974>.
- A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023.
- M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.
- D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, et al. A study of bfloat16 for deep learning training. arXiv preprint arXiv:1905.12322, 2019.
- S. Krishna, K. Krishna, A. Mohananey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. CoRR, abs/2409.12941, 2024. doi: 10.48550/ARXIV.2409.12941. URL <https://doi.org/10.48550/arXiv.2409.12941>.
- T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. P. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for question answering research. Trans. Assoc. Comput. Linguistics, 7:452–466, 2019. doi: 10.1162/tac1_a_00276. URL https://doi.org/10.1162/tac1_a_00276.
- G. Lai, Q. Xie, H. Liu, Y. Yang, and E. H. Hovy. RACE: large-scale reading comprehension dataset from examinations. In M. Palmer, R. Hwa, and S. Riedel, editors, Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017, pages 785–794. Association for Computational Linguistics, 2017. doi: 10.18653/V1/D17-1082. URL <https://doi.org/10.18653/v1/d17-1082>.
- N. Lambert, V. Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, et al. Rewardbench: Evaluating reward models for language modeling. arXiv preprint arXiv:2403.13787, 2024.
- D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In 9th International Conference on Learning Representations, ICLR 2021. OpenReview.net, 2021. URL <https://openreview.net/forum?id=qrwe7XHTmYb>.

- Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. *arXiv preprint arXiv:2306.09212*, 2023.
- S. Li and T. Hoefler. Chimera: efficiently training large-scale neural networks with bidirectional pipelines. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21*, page 1–14. ACM, Nov. 2021. doi: 10.1145/3458817.3476145. URL <http://dx.doi.org/10.1145/3458817.3476145>.
- T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint arXiv:2406.11939*, 2024a.
- W. Li, F. Qi, M. Sun, X. Yi, and J. Zhang. Ccpm: A chinese classical poetry matching dataset, 2021.
- Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=1NdN7eXyb4>.
- B. Y. Lin. ZeroEval: A Unified Framework for Evaluating Language Models, July 2024. URL <https://github.com/WildEval/ZeroEval>.
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- S. Lundberg. The art of prompt design: Prompt boundaries and token healing, 2023. URL <https://towardsdatascience.com/the-art-of-prompt-design-prompt-boundaries-and-token-healing-3b2448b0be38>.
- Y. Luo, Z. Zhang, R. Wu, H. Liu, Y. Jin, K. Zheng, M. Wang, Z. He, G. Hu, L. Chen, et al. Ascend HiFloat8 format for deep learning. *arXiv preprint arXiv:2409.16626*, 2024.
- MAA. American invitational mathematics examination - aime. In *American Invitational Mathematics Examination - AIME 2024*, February 2024. URL <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>.
- P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
- Mistral. Cheaper, better, faster, stronger: Continuing to push the frontier of ai and making it accessible to all, 2024. URL <https://mistral.ai/news/mixtral-8x22b>.
- S. Narang, G. Diamos, E. Elsen, P. Micikevicius, J. Alben, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, et al. Mixed precision training. In *Int. Conf. on Learning Representation*, 2017.

- B. Nouné, P. Jones, D. Justus, D. Masters, and C. Luschi. 8-bit numerical formats for deep neural networks. arXiv preprint arXiv:2206.02915, 2022.
- NVIDIA. Improving network performance of HPC systems using NVIDIA Magnum IO NVSH-MEM and GPUDirect Async. <https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async>, 2022.
- NVIDIA. Blackwell architecture. <https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/>, 2024a.
- NVIDIA. TransformerEngine, 2024b. URL <https://github.com/NVIDIA/TransformerEngine>. Accessed: 2024-11-19.
- OpenAI. Hello GPT-4o, 2024a. URL <https://openai.com/index/hello-gpt-4o/>.
- OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024b. URL <https://huggingface.co/datasets/openai/MMMLU>.
- OpenAI. Introducing SimpleQA, 2024c. URL <https://openai.com/index/introducing-simpleqa/>.
- OpenAI. Introducing SWE-bench verified we’re releasing a human-validated subset of swe-bench that more, 2024d. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023a.
- H. Peng, K. Wu, Y. Wei, G. Zhao, Y. Yang, Z. Liu, Y. Xiong, Z. Yang, B. Ni, J. Hu, et al. FP8-LM: Training FP8 large language models. arXiv preprint arXiv:2310.18313, 2023b.
- P. Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism. arXiv preprint arXiv:2401.10241, 2023a.
- P. Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism, 2023b. URL <https://arxiv.org/abs/2401.10241>.
- Qwen. Qwen technical report. arXiv preprint arXiv:2309.16609, 2023.
- Qwen. Introducing Qwen1.5, 2024a. URL <https://qwenlm.github.io/blog/qwen1.5>.
- Qwen. Qwen2.5: A party of foundation models, 2024b. URL <https://qwenlm.github.io/blog/qwen2.5>.
- S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.
- D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.
- B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, et al. Microscaling data formats for deep learning. arXiv preprint arXiv:2310.10537, 2023a.

- B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, et al. Microscaling data formats for deep learning. arXiv preprint arXiv:2310.10537, 2023b.
- K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In 5th International Conference on Learning Representations, ICLR 2017. OpenReview.net, 2017. URL <https://openreview.net/forum?id=B1ckMDqlg>.
- F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL <https://openreview.net/forum?id=fR3wGck-IXp>.
- Y. Shibata, T. Kida, S. Fukamachi, M. Takeda, A. Shinohara, T. Shinohara, and S. Arikawa. Byte pair encoding: A text compression scheme that accelerates pattern matching. 1999.
- J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.
- K. Sun, D. Yu, D. Yu, and C. Cardie. Investigating prior knowledge for challenging chinese machine reading comprehension, 2019a.
- M. Sun, X. Chen, J. Z. Kolter, and Z. Liu. Massive activations in large language models. arXiv preprint arXiv:2402.17762, 2024.
- X. Sun, J. Choi, C.-Y. Chen, N. Wang, S. Venkataramani, V. V. Srinivasan, X. Cui, W. Zhang, and K. Gopalakrishnan. Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks. Advances in neural information processing systems, 32, 2019b.
- M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.
- V. Thakkar, P. Ramani, C. Cecka, A. Shivam, H. Lu, E. Yan, J. Kosaian, M. Hoemmen, H. Wu, A. Kerr, M. Nicely, D. Merrill, D. Blasig, F. Qiao, P. Majcher, P. Springer, M. Hohnerbach, J. Wang, and M. Gupta. CUTLASS, Jan. 2023. URL <https://github.com/NVIDIA/cutlass>.
- H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023a.
- H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini,

- R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023b. doi: 10.48550/arXiv.2307.09288. URL <https://doi.org/10.48550/arXiv.2307.09288>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *CoRR*, abs/2408.15664, 2024a. URL <https://doi.org/10.48550/arXiv.2408.15664>.
- Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *CoRR*, abs/2406.01574, 2024b. URL <https://doi.org/10.48550/arXiv.2406.01574>.
- T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.
- M. Wortsman, T. Dettmers, L. Zettlemoyer, A. Morcos, A. Farhadi, and L. Schmidt. Stable and low-precision training for large-scale vision-language models. *Advances in Neural Information Processing Systems*, 36:10271–10298, 2023.
- H. Xi, C. Li, J. Chen, and J. Zhu. Training transformers with 4-bit integers. *Advances in Neural Information Processing Systems*, 36:49146–49168, 2023.
- C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint*, 2024.
- H. Xia, T. Ge, P. Wang, S. Chen, F. Wei, and Z. Sui. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, pages 3909–3925. Association for Computational Linguistics, 2023. URL <https://doi.org/10.18653/v1/2023.findings-emnlp.257>.
- G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023.
- L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020*, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL <https://doi.org/10.18653/v1/2020.coling-main.419>.

- R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL <https://doi.org/10.18653/v1/p19-1472>.
- W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL <https://doi.org/10.48550/arXiv.2304.06364>.
- J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023.

Appendix

A. Contributions and Acknowledgments

Research & Engineering

Aixin Liu	Lecong Zhang
Bing Xue	Liang Zhao
Bingxuan Wang	Litong Wang
Bochao Wu	Liyue Zhang
Chengda Lu	Mingchuan Zhang
Chenggang Zhao	Minghua Zhang
Chengqi Deng	Minghui Tang
Chenyu Zhang*	Panpan Huang
Chong Ruan	Peiyi Wang
Damai Dai	Qiancheng Wang
Daya Guo	Qihao Zhu
Dejian Yang	Qinyu Chen
Deli Chen	Qiushi Du
Erhang Li	Ruiqi Ge
Fangyun Lin	Ruisong Zhang
Fucong Dai	Ruizhe Pan
Fuli Luo*	Runji Wang
Guangbo Hao	Runxin Xu
Guanting Chen	Ruoyu Zhang
Guowei Li	Shanghao Lu
H. Zhang	Shangyan Zhou
Han Bao*	Shanhuang Chen
Hanwei Xu	Shengfeng Ye
Haocheng Wang*	Shirong Ma
Haowei Zhang	Shiyu Wang
Honghui Ding	Shuiping Yu
Huajian Xin*	Shunfeng Zhou
Huazuo Gao	Shuting Pan
Hui Qu	Tao Yun
Jianzhong Guo	Tian Pei
Jiashi Li	Wangding Zeng
Jiawei Wang*	Wanjia Zhao*
Jingchang Chen	Wen Liu
Jingyang Yuan	Wenfeng Liang
Junjie Qiu	Wenjun Gao
Junlong Li	Wenqin Yu
Junxiao Song	Wentao Zhang
Kai Dong	Xiao Bi
Kai Hu*	Xiaodong Liu
Kaige Gao	Xiaohan Wang
Kang Guan	Xiaokang Chen
Kexin Huang	Xiaokang Zhang
Kuai Yu	Xiaotao Nie
Lean Wang	Xin Cheng
	Xin Liu

Xin Xie
Xingchao Liu
Xingkai Yu
Xinyu Yang
Xinyuan Li
Xuecheng Su
Xuheng Lin
Y.K. Li
Y.Q. Wang
Y.X. Wei
Yang Zhang
Yanhong Xu
Yao Li
Yao Zhao
Yaofeng Sun
Yaohui Wang
Yi Yu
Yichao Zhang
Yifan Shi
Yiliang Xiong
Ying He
Yishi Piao
Yisong Wang
Yixuan Tan
Yiyang Ma*
Yiyuan Liu
Yongqiang Guo
Yu Wu
Yuan Ou
Yuduan Wang
Yue Gong
Yuheng Zou
Yujia He
Yunfan Xiong
Yuxiang Luo
Yuxiang You
Yuxuan Liu
Yuyang Zhou
Z.F. Wu
Z.Z. Ren
Zehui Ren
Zhangli Sha
Zhe Fu
Zhean Xu
Zhenda Xie
Zhengyan Zhang
Zhewen Hao
Zhibin Gou
Zhicheng Ma

Zhigang Yan
Zhihong Shao
Zhiyu Wu
Zhuoshu Li
Zihui Gu
Zijia Zhu
Zijun Liu*
Zilin Li
Ziwei Xie
Ziyang Song
Ziyi Gao
Zizheng Pan

Data Annotation

Bei Feng
Hui Li
J.L. Cai
Jiaqi Ni
Lei Xu
Meng Li
Ning Tian
R.J. Chen
R.L. Jin
Ruyi Chen
S.S. Li
Shuang Zhou
Tianyu Sun
X.Q. Li
Xiangyue Jin
Xiaojin Shen
Xiaosha Chen
Xiaowen Sun
Xiaoxiang Wang
Xinnan Song
Xinyi Zhou
Y.X. Zhu
Yanhong Xu
Yanping Huang
Yaohui Li
Yi Zheng
Yuchen Zhu
Yunxian Ma
Zhen Huang
Zhipeng Xu
Zhongyu Zhang

Business & Compliance

Dongjie Ji

Jian Liang
 Jin Chen
 Leyi Xia
 Miaojun Wang
 Mingming Li
 Peng Zhang
 Shaoqing Wu
 Shengfeng Ye
 T. Wang

W.L. Xiao
 Wei An
 Xianzu Wang
 Xinxia Shan
 Ying Tang
 Yukun Zha
 Yuting Yan
 Zhen Zhang

Within each role, authors are listed alphabetically by the first name. Names marked with * denote individuals who have departed from our team.

B. Ablation Studies for Low-Precision Training

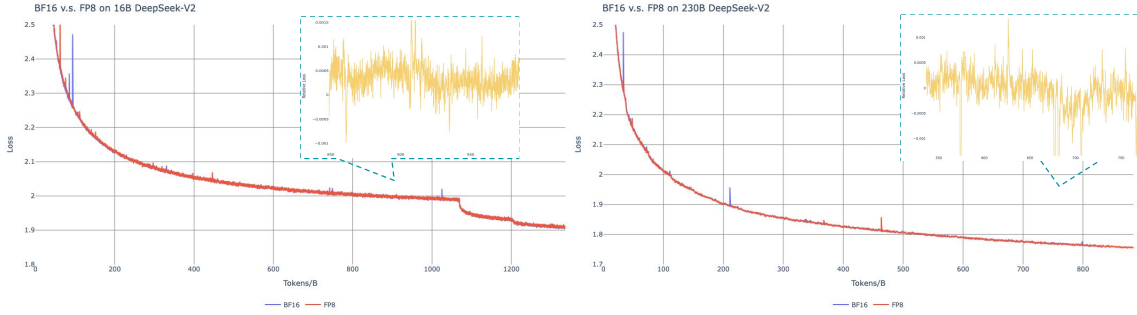


Figure 10 | Loss curves comparison between BF16 and FP8 training. Results are smoothed by Exponential Moving Average (EMA) with a coefficient of 0.9.

B.1. FP8 v.s. BF16 Training

We validate our FP8 mixed precision framework with a comparison to BF16 training on top of two baseline models across different scales. At the small scale, we train a baseline MoE model comprising approximately 16B total parameters on 1.33T tokens. At the large scale, we train a baseline MoE model comprising approximately 230B total parameters on around 0.9T tokens. We show the training curves in Figure 10 and demonstrate that the relative error remains below 0.25% with our high-precision accumulation and fine-grained quantization strategies.

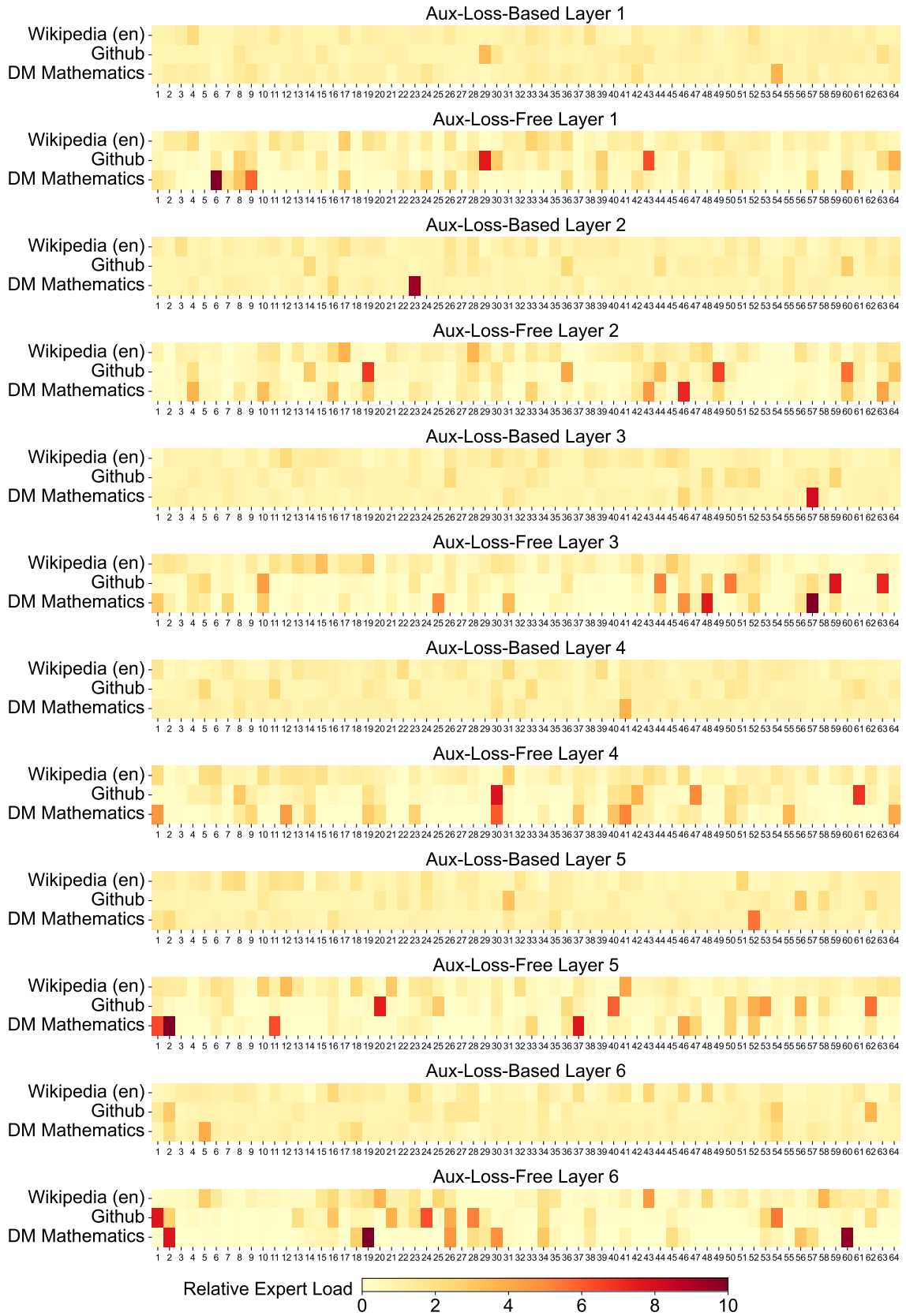
B.2. Discussion About Block-Wise Quantization

Although our tile-wise fine-grained quantization effectively mitigates the error introduced by feature outliers, it requires different groupings for activation quantization, i.e., 1×128 in forward pass and 128×1 for backward pass. A similar process is also required for the activation gradient. A straightforward strategy is to apply block-wise quantization per 128×128 elements like the way we quantize the model weights. In this way, only transposition is required for backward. Therefore, we conduct an experiment where all tensors associated with Dgrad are quantized on a block-wise basis. The results reveal that the Dgrad operation which computes the activation gradients and back-propagates to shallow layers in a chain-like manner, is highly sensitive to precision. Specifically, block-wise quantization of activation gradients leads to

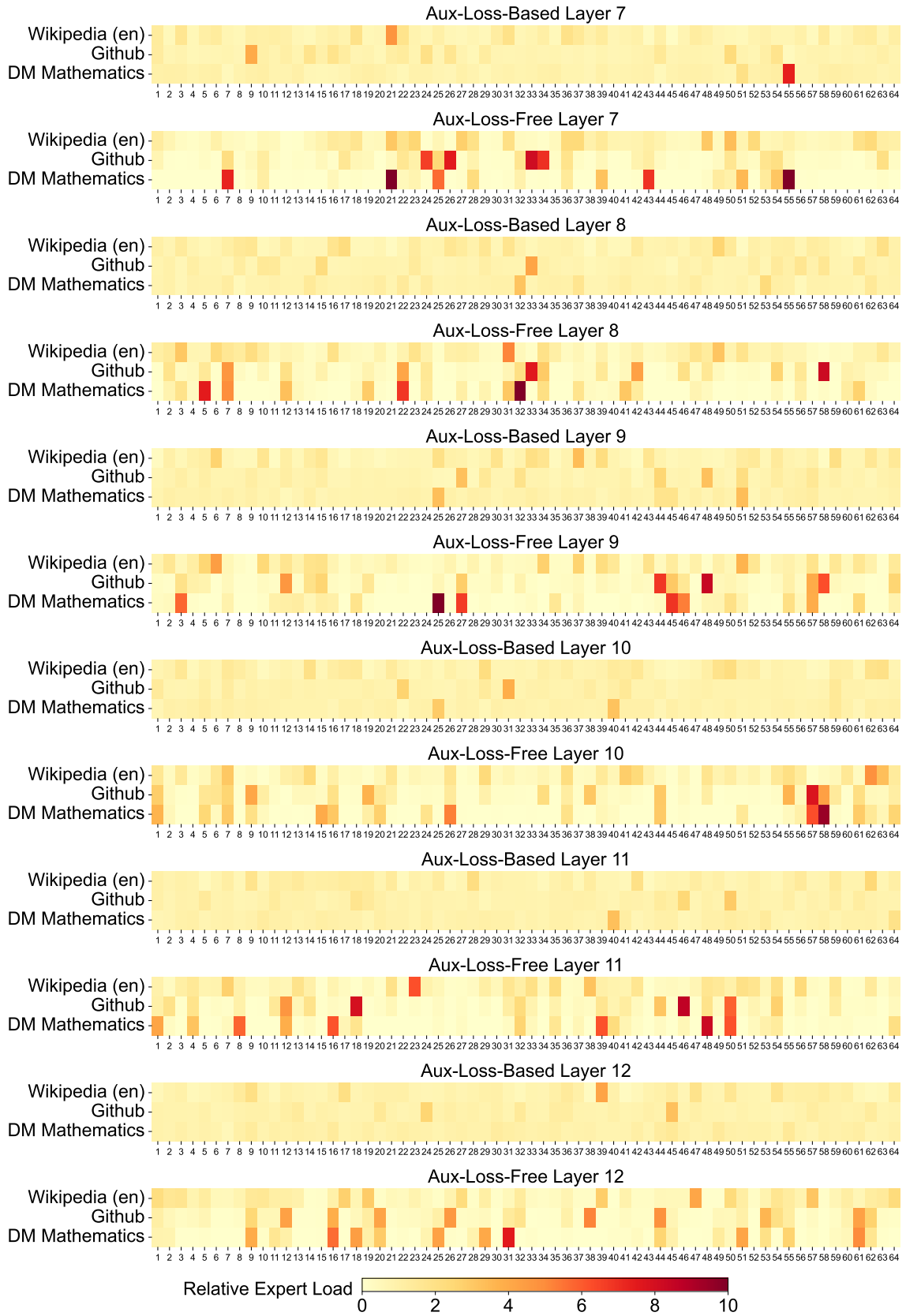
model divergence on an MoE model comprising approximately 16B total parameters, trained for around 300B tokens. We hypothesize that this sensitivity arises because activation gradients are highly imbalanced among tokens, resulting in token-correlated outliers (Xi et al., 2023). These outliers cannot be effectively managed by a block-wise quantization approach.

C. Expert Specialization Patterns of the 16B Aux-Loss-Based and Aux-Loss-Free Models

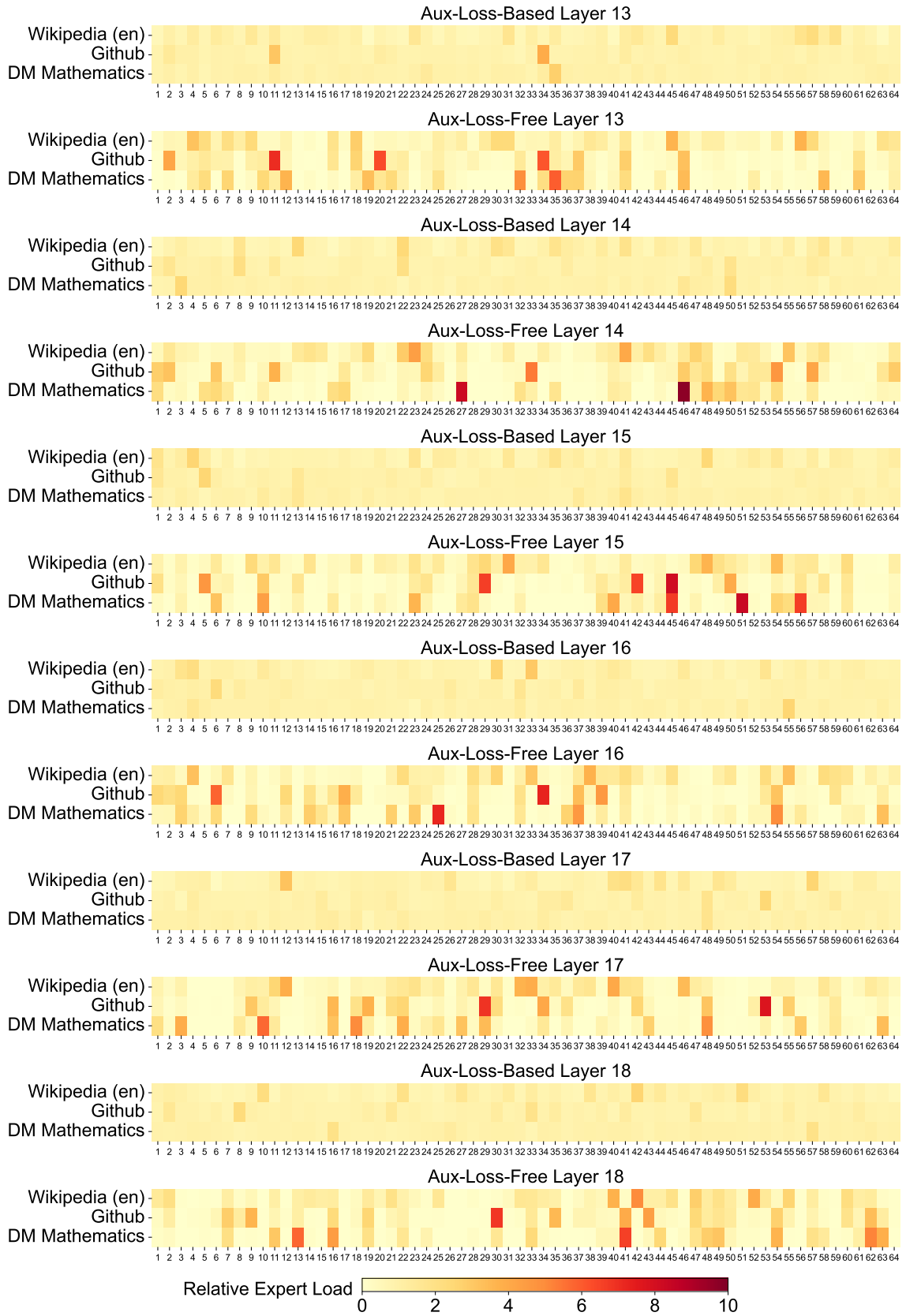
We record the expert load of the 16B auxiliary-loss-based baseline and the auxiliary-loss-free model on the Pile test set. The auxiliary-loss-free model tends to have greater expert specialization across all layers, as demonstrated in Figure 10.



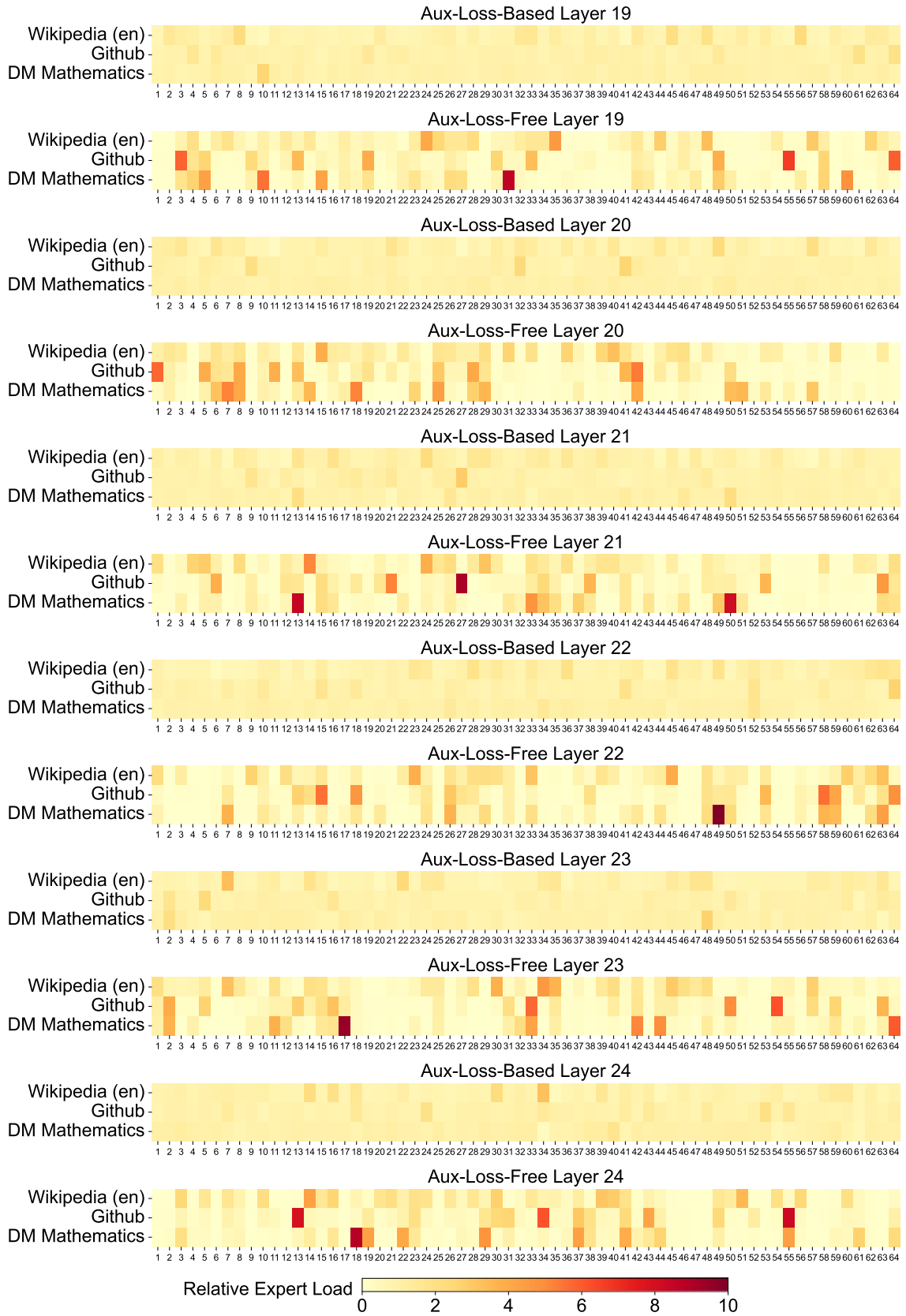
(a) Layers 1-7



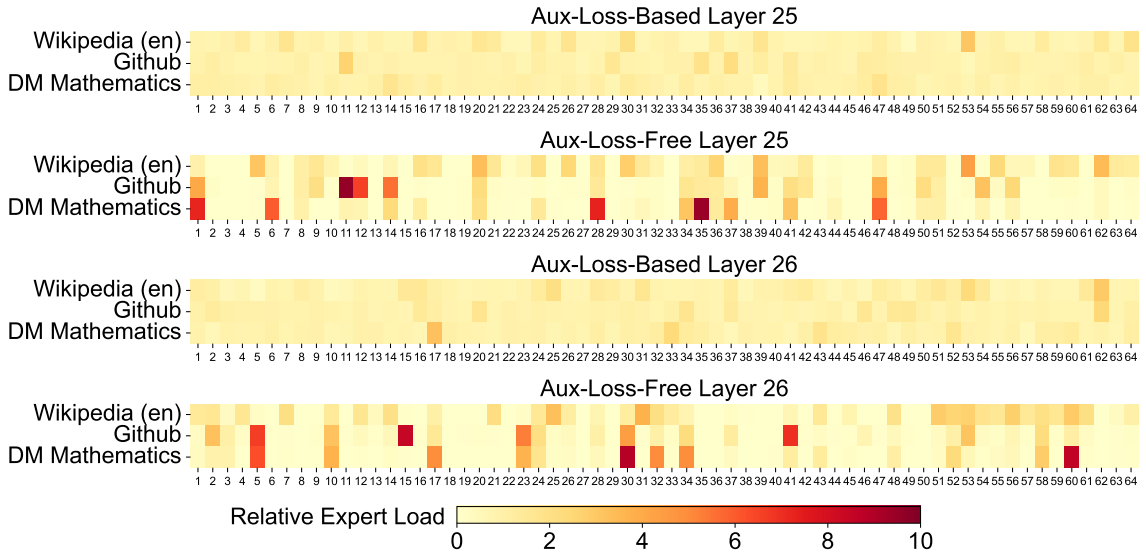
(b) Layers 7-13



(c) Layers 13-19



(d) Layers 19-25



(e) Layers 25-27

Figure 10 | Expert load of auxiliary-loss-free and auxiliary-loss-based models on three domains in the Pile test set. The auxiliary-loss-free model shows greater expert specialization patterns than the auxiliary-loss-based one. The relative expert load denotes the ratio between the actual expert load and the theoretically balanced expert load.